

BỘ GIÁO DỤC VÀ ĐÀO TẠO

ĐẠI HỌC UEH – TRƯỜNG CÔNG NGHỆ VÀ THIẾT KẾ

KHOA CÔNG NGHỆ THÔNG TIN KINH DOANH

CHUYÊN NGÀNH HỆ THỐNG THÔNG TIN QUẢN LÝ



KHÓA LUẬN TỐT NGHIỆP

**SPO (SMART PREDICT-THEN-OPTIMIZE) TRONG
EMAIL MARKETING: TỐI ƯU HÓA LỰA CHỌN
CHIẾN DỊCH TIẾP THỊ TỪ VIỆC DỰ BÁO CỦA
CÁC MÔ HÌNH HỌC MÁY VÀ ĐÁNH GIÁ
TỶ LỆ THẤT KINH TẾ CỦA TỪNG MÔ HÌNH
DỰA TRÊN HÀM MẤT MẤT SPO LOSS**

Họ và tên sinh viên: Nguyễn Trần Minh Nhật

Mã số sinh viên: 31221023438

Lớp: BI002 – **Khóa:** 48

Giáo viên hướng dẫn: TS. Huỳnh Văn Đức

Niên khóa: 2022 – 2025

TP. Hồ Chí Minh, ngày 15 tháng 10 năm 2025

BỘ GIÁO DỤC VÀ ĐÀO TẠO

ĐẠI HỌC UEH – TRƯỜNG CÔNG NGHỆ VÀ THIẾT KẾ

KHOA CÔNG NGHỆ THÔNG TIN KINH DOANH

CHUYÊN NGÀNH HỆ THỐNG THÔNG TIN QUẢN LÝ



KHÓA LUẬN TỐT NGHIỆP

**SPO (SMART PREDICT-THEN-OPTIMIZE) TRONG
EMAIL MARKETING: TỐI ƯU HÓA LỰA CHỌN
CHIẾN DỊCH TIẾP THỊ TỪ VIỆC DỰ BÁO CỦA
CÁC MÔ HÌNH HỌC MÁY VÀ ĐÁNH GIÁ
TỶ LỆ THẤT KINH TẾ CỦA TỪNG MÔ HÌNH
DỰA TRÊN HÀM MẤT MẤT SPO LOSS**

Họ và tên sinh viên: Nguyễn Trần Minh Nhật

Mã số sinh viên: 31221023438

Lớp: BI002 – **Khóa:** 48

Giáo viên hướng dẫn: TS. Huỳnh Văn Đức

Niên khóa: 2022 – 2025

TP. Hồ Chí Minh, ngày 15 tháng 10 năm 2025

TÓM TẮT ĐỀ TÀI

Trong bối cảnh doanh nghiệp triển khai nhiều chiến dịch email marketing với một bộ dữ liệu gồm rất nhiều chiến dịch với đặc trưng của email (độ dài tiêu đề, độ dài nội dung, số lượng CTA), các đặc trưng về ngữ cảnh (thời gian gửi, ngày trong tuần,...) và các đặc trưng liên quan đến sản phẩm, ta ước lượng/ dự báo tỷ lệ nhấp chuột (CTR) cũng như tối ưu hóa lựa chọn tập chiến dịch có được coi là hiệu quả và đưa về giá trị kinh tế cao.

Khóa luận triển khai khung SPO (Smart Predict-then-Optimize) với tầng ước lượng/ dự báo (Predict/ Estimate Layer) bằng cách ứng dụng các mô hình học máy hồi quy tiêu biểu như k-NN, Random Forest, SVM, Linear Regression sau đó đánh giá chỉ số được dự báo trên các chỉ số cơ bản (RMSE, MSE, MAE) bằng Orange và các thư viện Python. Từ việc ước lượng/ dự báo đó, CTR sẽ là đầu vào cho bài toán tối ưu danh định ở tầng tối ưu hóa (Optimize Layer), sử dụng phương pháp MILP (Mixed-Integer Linear Programming) và Knapsack lựa chọn tập chiến dịch tối ưu, tuân thủ ràng buộc ngân sách và hạn mức (quota) trong khung giờ gửi. Điểm nhấn của nghiên cứu là sử dụng chỉ số SPO Loss (Smart Predict-then-Optimize Loss) để đo lường hiệu quả của việc ra quyết định sau tối ưu bằng cách tính toán giá trị tổn thất kinh tế.

Kết quả thực nghiệm đã cho ra một tập các chiến dịch tối ưu của từng mô hình và chỉ ra rằng Random Forest là mô hình tốt nhất trong 4 mô hình học máy kể trên trong việc đưa ra ước lượng/ dự báo rồi tối ưu hóa tập chiến dịch hiệu quả hơn, đồng thời đưa ra các quyết định hậu tối ưu mang lại ít giá trị tổn thất hơn với SPO Loss thấp nhất. Từ đó doanh nghiệp có thể tham khảo những chiến dịch được các đặc trưng email từ đó cải thiện chất lượng email để thu hút CTR nhiều hơn, giảm thiểu được tổn thất kinh tế.

Từ khóa: Email Marketing, CTR, MILP, SPO, SPO Loss, Machine Learning.

LỜI CẢM ƠN

Lời đầu tiên, em xin cảm ơn UEH đã tạo điều kiện tốt nhất cho em và các bạn sinh viên trong suốt gần 4 năm học Đại học. UEH có môi trường học tập uy tín và lâu đời, khuyến khích sinh viên phát triển toàn diện cả về kiến thức, kỹ năng mà còn hướng người học phát triển tư duy đổi mới sáng tạo, phát triển bền vững.

Xin trân trọng cảm ơn chân thành nhất tới các thầy thuộc Khoa Công nghệ thông tin kinh doanh đã chỉ bảo tận tình, đưa đến các kiến thức bộ môn, giúp sinh viên rèn luyện được tư duy về hệ thống, dữ liệu và những vấn đề liên quan đến ngành Hệ thống thông tin quản lý.

Em cũng xin cảm ơn thầy Huỳnh Văn Đức đã giúp đỡ em và các bạn trong nhóm làm đề tài tốt nghiệp cho đợt Thực tập tốt nghiệp này. Suốt 2 tháng vừa qua, mặc dù có đôi lúc khó khăn trong việc tìm hướng nghiên cứu và phát triển đề tài, thầy đã hướng dẫn một cách dễ hiểu và gần gũi nhất về cách xây dựng đề tài và tư duy thực hiện đề tài một cách chuẩn chỉnh theo hướng Nghiên cứu khoa học. Sự tận tâm và định hướng nghiên cứu của thầy đóng vai trò rất lớn trong việc hoàn thành Khóa luận tốt nghiệp của chúng em trong kỳ Thực tập tốt nghiệp lần này.

MỤC LỤC

TÓM TẮT ĐỀ TÀI.....	i
LỜI CẢM ƠN	ii
MỤC LỤC	iii
DANH MỤC CÁC CHỮ VIẾT TẮT VÀ KÝ HIỆU	vi
DANH MỤC HÌNH ẢNH VÀ BẢNG BIỂU	vii
CHƯƠNG 1: TỔNG QUAN	1
1.1. Lý do chọn đề tài	1
1.2. Mục tiêu nghiên cứu	1
1.3. Đối tượng và phạm vi nghiên cứu	1
1.4. Phương pháp nghiên cứu	2
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT	3
2.1. Email Marketing và chiến dịch tiếp thị.	3
2.2. Click-through rate (CTR)	3
2.3. Phân tích hồi quy, hồi quy tuyến tính và hồi quy phi tuyến.	4
2.4. Tiền xử lý dữ liệu	5
2.4.1. Làm sạch dữ liệu.....	5
2.4.2. Biến đổi dữ liệu	5
2.4.3. One-hot encoding	6
2.5. Áp dụng và đánh giá các mô hình học máy hồi quy trong ước lượng/ dự báo	6
2.5.1. k-NN	6
2.5.2. Random Forest.....	7
2.5.3. SVM và SVR.....	7
2.5.4. Linear Regression	8

2.5.5. Các chỉ số đánh giá mô hình.....	9
2.5.6. Kiểm định chéo (Cross-validation)	9
2.5.7. Ước lượng/ Dự báo bằng Orange	10
2.5.8. Ước lượng/ Dự báo bằng thư viện Python SciKit-Learn	10
2.6. Các vấn đề và phương pháp tối ưu hóa	11
2.6.1. Tối ưu hóa danh định (Nominal Optimization Problem)	11
2.6.2. MILP (Mixed-Integer Linear Programming)	12
2.6.3. Thuật toán “tham lam” Knapsack (Greedy Knapsack)	12
2.6.4. Thư viện Python dành cho tối ưu hóa SciPy.optimize.milp.....	13
2.7. Khung SPO (Smart Predict-then-Optimize), hàm mất mát (SPO Loss) và mô hình hóa bài toán SPO trong Email Marketing.....	15
2.7.1. Thiết kế mô hình tích hợp SPO (ước lượng/ dự báo rồi tối ưu hóa) trong Email Marketing và đánh giá bằng SPO Loss.....	16
2.7.2. Tầng ước lượng/ dự báo (Estimate/ Predict)	17
2.7.3. Tầng tối ưu (Optimize).....	17
2.7.4. Hàm mục tiêu và các ràng buộc phục vụ cho bài toán tối ưu.....	18
2.7.5. Hàm mất mát SPO Loss	21
2.7.6. Trực quan hóa SPO Loss bằng thư viện Python Matplotlib.....	23
CHƯƠNG 3: DỮ LIỆU, CÁC PHƯƠNG PHÁP HIỆN THỰC BÀI TOÁN VÀ THỰC NGHIỆM	24
3.1. Giới thiệu bộ dữ liệu được sử dụng và các đặc trưng của dữ liệu.....	24
3.1.1. Tổng quan về dữ liệu	24
3.1.2. Độ phân tán dữ liệu	26
3.1.3. Thuộc tính (Features) được chọn.....	27
3.1.4. Biến target (Biến mục tiêu)	28

3.2. Ước lượng/ dự báo.....	28
3.2.1. Ước lượng/ Dự báo bằng Orange	28
3.2.2. Dự báo bằng Python với thư viện Scikit-learn	29
3.3. Tối ưu hóa bằng thư viện Python MILP	36
3.4. Đo lường chỉ số SPO Loss.....	41
CHƯƠNG 4: KẾT QUẢ THỰC NGHIỆM	48
4.1. So sánh hiệu quả của các mô hình học máy và thực hiện ước lượng/ dự báo	48
4.1.1. Ước lượng/ dự báo với công cụ Orange	48
4.1.2. Dự báo bằng cách sử dụng thư viện Python Scikit-Learn.....	52
4.2. Kết quả lựa chọn chiến dịch sau tối ưu hóa từ dự báo/ ước lượng	55
4.3. Phân tích hiệu quả quyết định bằng hàm mất mát SPO Loss	58
4.3.1. Nhận xét trên từng fold.....	59
4.3.2. Nhận xét trên toàn cục	60
4.3.3. Kết quả trực quan hóa so sánh SPO Loss trên từng mô hình	60
CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	62
5.1. Đóng góp của đề tài	62
5.2. Hạn chế và hướng phát triển.....	62
5.2.1. Hạn chế	62
5.2.2. Hướng phát triển.....	63
TÀI LIỆU THAM KHẢO.....	64
PHỤ LỤC A: HƯỚNG DẪN SỬ DỤNG BỘ DỮ LIỆU KAGGLE	67
PHỤ LỤC B: HƯỚNG DẪN TÁI LẬP THỰC NGHIỆM BẰNG PYCHARM ...	68

DANH MỤC CÁC CHỮ VIẾT TẮT VÀ KÝ HIỆU

Từ viết tắt	Từ đầy đủ
CTA	Call To Action
CTR	Click-through Rate
CV	Cross Validation
k-NN	k-Nearest Neighbors
MAE	Mean Absolute Error
MAPE	Mean Absolute Percentage Error
MILP	Mixed-Integer Linear Programming
ML	Machine Learning
MSE	Mean Squared Error
OOF	Out-of-fold
RMSE	Root Mean Squared Error
SPO	Smart Predict-then-Optimize
SVM	Support Vector Machines
SVR	Support Vector Regression

DANH MỤC HÌNH ẢNH VÀ BẢNG BIỂU

Hình 2.1. Giao diện Orange.....	10
Hình 2.2. Sơ đồ luồng làm việc của mô hình tích hợp SPO và đánh giá bằng SPO Loss	17
Hình 3.1. Bảng đặc trưng của tập dữ liệu train trong Orange	24
Hình 3.2. Bảng đặc trưng của dữ liệu test trong Orange	25
Hình 3.3. Thể hiện các dòng dữ liệu của train set	25
Hình 3.4. Thể hiện các dòng dữ liệu trong test set	26
Hình 3.5. Thể hiện các dòng dữ liệu của test set - không có click_rate	26
Hình 3.6. Biểu đồ thể hiện độ phân tán của dữ liệu	27
Hình 3.7. Workflow thực hiện dự báo trong Orange	28
Hình 4.1. Bảng kết quả đánh giá mô hình học máy với cross-validation k-fold = 5 ...	48
Hình 4.2. Bảng ước lượng CTR với nhiều mô hình khác nhau ở train set với k-fold = 5	49
Hình 4.3. Bảng đánh giá các mô hình học máy với cross-validation theo feature times_of_day	50
Hình 4.4. Bảng ước lượng CTR với các mô hình học máy khác nhau bằng cross- validation theo feature times_of_day ở train set	51
Hình 4.5. Bảng dự báo CTR với nhiều mô hình học máy trên test set.....	52
Hình 4.6. Bảng kết quả đánh giá các mô hình học máy trên Python	53
Hình 4.7. Bảng dự báo CTR và vector chi phí với mô hình k-NN	53
Hình 4.8. Bảng dự báo CTR và vector chi phí với mô hình Linear Regression	54
Hình 4.9. Bảng dự báo CTR và vector chi phí với mô hình Random Forest	54
Hình 4.10. Bảng dự báo CTR và vector chi phí với mô hình SVM.....	55
Hình 4.11. Cài đặt các ràng buộc bằng cách cho người dùng nhập input	55

Hình 4.12. Kết quả tổng quan từ tối ưu tập chiến dịch	56
Hình 4.13. Bảng thể hiện tập các chiến dịch tối ưu được chọn sau khi dự báo bằng mô hình Linear Regression	56
Hình 4.14. Bảng thể hiện tập các chiến dịch tối ưu được chọn sau khi dự báo bằng mô hình Random Forest.....	57
Hình 4.15. Bảng thể hiện tập các chiến dịch tối ưu được chọn sau khi dự báo bằng mô hình SVM	57
Hình 4.16. Bảng thể hiện tập các chiến dịch tối ưu sau khi dự báo bằng mô hình k-NN	58
Hình 4.17. Bảng đo lường SPO Loss với từng mô hình trên từng fold.....	59
Hình 4.18. Bảng đo lường SPO Loss với từng mô hình trên toàn cục	60
Hình 4.19: Biểu đồ cột thể hiện so sánh chỉ số SPO Loss trung bình theo mô hình học máy (error bar = Std SPO Loss)	61
Hình A.1. Giao diện trang web chứa bộ dữ liệu Kaggle	67

CHƯƠNG 1: TỔNG QUAN

1.1. Lý do chọn đề tài

Trong bối cảnh các doanh nghiệp ngày càng chú trọng trong việc chuyển đổi số và sự tham gia của các công nghệ mới như AI, Email Marketing vẫn là một kênh truyền thông truyền thống nhưng hiệu quả, không phân biệt quy mô của doanh nghiệp. Tuy nhiên, việc triển khai nhiều chiến dịch gửi Email cùng lúc khiến cho doanh nghiệp luôn gặp khó khăn trong việc đánh giá chiến dịch nào thật sự là chiến dịch tối ưu nhằm ra quyết định cho những chiến dịch gửi Email tiếp theo với một bộ dữ liệu về kết quả của những chiến dịch Email Marketing.

Vì thế, việc đưa bộ dữ liệu vào các mô hình học máy trong dự báo sau đó đưa ra quyết định tối ưu cho từng chiến dịch giúp doanh nghiệp phân bổ được nguồn lực, cải thiện đặc trưng của email nhằm giảm chi phí đầu tư vào email, từ đó tăng lợi nhuận cho doanh nghiệp.

1.2. Mục tiêu nghiên cứu

- So sánh và đánh giá các mô hình học máy nào được xem là tốt bằng phương pháp truyền thống qua các chỉ số RMSE, MAE, MSE và ước lượng/ dự báo được CTR với độ sai số nhỏ.
- Lựa chọn ra được các tập chiến dịch hiệu quả từ bài toán tối ưu qua việc dự báo của từng mô hình học máy.
- Tìm ra mô hình học máy nào có giá trị tổn thất kinh tế thấp nhất từ việc tối ưu dự báo qua phương pháp đánh giá mô hình bằng chỉ số SPO Loss có giá trị trung bình trên từng phần và trên toàn cục thấp nhất.

1.3. Đối tượng và phạm vi nghiên cứu

- **Đối tượng nghiên cứu:** Bộ dữ liệu các chiến dịch Email Marketing đã có sẵn các đặc trưng của email, ngữ cảnh và đặc trưng về sản phẩm.
- **Phạm vi nghiên cứu:**

- Tập trung vào việc huấn luyện mô hình học máy để dự báo chỉ số, là đầu vào cho công việc tối ưu hóa chiến dịch dựa trên dữ liệu có sẵn.
- Đo lường chỉ số SPO Loss để đánh giá hiệu quả của quyết định tối ưu từ dự báo.
- Bộ dữ liệu được sử dụng là bộ dữ liệu Email CTR Prediction từ Kaggle.

1.4. Phương pháp nghiên cứu

- **Phương pháp thu thập dữ liệu:** Sử dụng bộ dữ liệu công khai từ Kaggle
- **Phương pháp tiền xử lý dữ liệu:** chọn lọc đặc trưng, chuẩn hóa, xử lý thiếu dữ liệu, mã hóa biến phân loại.
- **Phương pháp đánh giá, ước lượng/ dự báo:** So sánh các mô hình dự báo bằng cách đo lường các chỉ số RMSE, MSE, R2; Đánh giá quyết định sau tối ưu bằng chỉ số SPO Loss.
- **Phương pháp tối ưu hóa:** sử dụng bộ thư viện Python SciPy phục vụ cho việc tối ưu hóa (MILP), xét ràng buộc về ngân sách, chi phí có thể bỏ ra, hạn mức (quota) theo khung giờ.
- **Phương pháp lập trình:** Xây dựng hàm Knapsack và tính toán SPO Loss trên Python.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. Email Marketing và chiến dịch tiếp thị.

Email Marketing có rất nhiều định nghĩa khác nhau, theo định nghĩa của Wu et al (2018), Email Marketing là một hình thức quảng cáo trực tiếp bằng cách sử dụng Email. Trong khi đó Bawm & Nath (2014) định nghĩa Email Marketing là việc sử dụng email nhằm truyền tải các tin nhắn thương mại hóa tới một nhóm người.

Chiến dịch tiếp thị (campaign) tức các hoạt động nhằm tới quảng bá thương hiệu, sản phẩm tới khách hàng. Mục đích lan truyền rộng rãi thông qua nhiều phương tiện khác nhau, tiếp cận và thu hút nhiều khách hàng. Chiến dịch marketing rất quan trọng bởi quyết định tới sự thành công của cả một nhãn hàng/thương hiệu/doanh nghiệp. (GOBRANDING, 2025).

Khi triển khai một chiến dịch gửi email điều đầu tiên ta cần xem xét là **CTA (Call To Action)** - một đặc trưng quan trọng trong email. Trong một bài báo của Mejtoft et al. (2021) đã khẳng định rằng CTA là một công cụ được sử dụng trong Marketing nhằm thúc đẩy sự phản hồi ngay lập tức hoặc khuyến khích hoạt động. CTA có thể là một trong những yếu tố quyết định khiến cho người dùng có quyết định hành động đúng như doanh nghiệp mong muốn hay không. CTA giúp doanh nghiệp kiếm được lợi nhuận từ việc lôi kéo khách hàng mua sản phẩm, đồng thời cũng là một thước đo sử dụng trong việc phân tích, định lượng.

2.2. Click-through rate (CTR)

Trong một chiến dịch Email, yếu tố then chốt của doanh nghiệp luôn quan tâm là liệu với số lượng email đã gửi của chiến dịch liệu có khiến cho khách hàng để ý và nhấp vào đường link để tìm hiểu hơn về sản phẩm hay không. Click-through rate là một thang đo giúp doanh nghiệp hiểu rõ hơn về độ thu hút của các chiến dịch Email đó.

CTR (Click-through rate) là tỉ lệ người dùng nhấp chuột vào ít nhất một link/ CTA và số lượng email đã gửi.

$$CTR = \frac{\text{Số lượt click}}{\text{Số email được gửi hoặc hiển thị}}$$

Số lượt click: số lần người nhận email nhấp vào link/CTA trong email.

Số email được gửi hoặc hiển thị: tổng số email gửi thành công (delivered).

CTR là một **tỉ lệ đã được chuẩn hóa trong dataset** và luôn nằm trong khoảng $[0,1]$

Các đặc trưng về email, trong đó có số lượng và chất lượng của CTA đều có thể ảnh hưởng lên quyết định của người dùng nhấp chuột, từ đó thay đổi giá trị CTR.

2.3. Phân tích hồi quy, hồi quy tuyến tính và hồi quy phi tuyến.

Phân tích hồi quy bao gồm nhiều kỹ thuật mô hình hóa mối quan hệ giữa các biến độc lập (hay còn được gọi là các biến phản hồi) và một hay nhiều biến phụ thuộc (được gọi là các biến giải thích hoặc biến dự báo). Trong hồi quy, biến độc lập là một hàm của một tập các tham số đi kèm với tham số ngẫu nhiên (Abebe, 2024).

Phương trình hồi quy tổng quát (population regression function) có dạng:

$$Y_i = \beta_0 + \beta_1 X_i + \varepsilon_i, i = 1, 2, \dots, n$$

(Abebe, 2024)

Với β_0 và β_1 lần lượt là hệ số chặn và hệ số góc, hai tham số này cố định chưa biết, Y là biến phụ thuộc, X là biến độc lập, ε là một sai số ngẫu nhiên độc lập và phân phối đồng nhất đại diện cho các biến bị bỏ sót khỏi mô hình nhưng có ảnh hưởng đến biến phụ thuộc Y.

Đối với hồi quy tuyến tính, biến phụ thuộc là một hàm tuyến tính (nằm trên một đường thẳng) của tập các tham số hồi quy và một sai số ngẫu nhiên.

Trong khi đó, hồi quy phi tuyến mô tả mối quan hệ của biến phụ thuộc Y với một hay nhiều biến độc lập khác $x = (x_1, \dots, x_k)^T$, trong đó các tham số xuất hiện theo cách phi

tuyến (phân tán và không nằm trên một đường thẳng) trong hàm dự đoán. Khác với hồi quy tuyến tính (các hệ số xuất hiện một cách tuyến tính), hồi quy phi tuyến có dạng:

$$Y_i = f(x_i, \theta) + \varepsilon_i, i = 1, \dots, n$$

(El-Shaarawi & Piegorsch, 2002, pp. 1405–1411)

Với Y_i là giá trị phản hồi (responses); f là một hàm đã biết với $x_i = (x_{i1}, \dots, x_{ik})^T$ là vector biến độc lập, $\theta = (\theta_1, \dots, \theta_p)^T$ là vector tham số và ε_i là độ sai số.

Ta có thể xem CTR là một biến độc lập có tính liên tục và chịu ảnh hưởng của nhiều biến phụ thuộc khác là các đặc trưng của Email và các đặc trưng về sản phẩm.

2.4. Tiền xử lý dữ liệu

Để phục vụ cho bài toán dự báo hồi quy, ta thực hiện tiền xử lý đầu vào vì đây là giai đoạn quan trọng trong học máy, nhằm biến đổi dữ liệu thô thành dữ liệu “sạch” phục vụ cho việc huấn luyện ML. Han et al. (2012) chia các phương pháp dữ liệu thành 6 nhóm chính, tuy nhiên chúng ta sẽ chỉ quan tâm đến 2 nhóm phổ biến bao gồm:

2.4.1. Làm sạch dữ liệu

Có 2 phương pháp làm sạch dữ liệu bao gồm

Điền dữ liệu thiếu: Trong trường hợp một hay nhiều ô dữ liệu bị thiếu trên một thuộc tính thì tự động điền giá trị xung quanh giá trị trung vị hay giá trị trung bình của thuộc tính đang xét. Đối với biến phân loại thì tự động điền giá trị có tần suất xuất hiện nhiều nhất.

Xử lý dữ liệu nhiễu: làm “muột” bộ dữ liệu bằng cách phương pháp *binning*, *regression* (đưa về một đường thẳng), *outlier*.

2.4.2. Biến đổi dữ liệu

Chuẩn hóa dữ liệu (Normalization/ scaling): đưa giá trị các dòng dữ liệu của thuộc tính về cùng một thang đo.

Tổng hợp theo nhóm (Aggregation), VD: tính trung bình CTR theo ngày hoặc giờ.

Tạo thuộc tính mới (Attribute Construction) từ các thuộc tính sẵn có.

2.4.3. One-hot encoding

Để phục vụ cho việc dự báo CTR, ngoài những phương pháp tiền xử lý ở trên chúng ta sử dụng One - hot encoding đối với các biến phân loại - một phương pháp kỹ thuật biến đổi biến phân loại thành dạng biểu diễn nhị phân (0 hoặc 1) với mỗi giá trị được biểu diễn bởi số 1 ở vị trí tương ứng của nó và bởi số 0 ở tất cả các vị trí khác (Hancock & Khoshgoftaar, 2020).

Với đề tài, để phục vụ cho việc ước lượng/ dự báo được chính xác, các thuộc tính số học trong bộ dữ liệu sẽ được chuẩn hóa (normalization) về cùng một thang đo, các thuộc tính phân loại sẽ sử dụng One-hot encoding để đồng bộ thang đo. Các bước tiền xử lý dữ liệu này giúp đảm bảo sự ổn định của đầu vào, tránh sai số quá lớn từ các mô hình ML.

2.5. Áp dụng và đánh giá các mô hình học máy hồi quy trong ước lượng/ dự báo

Sau khi định hình được tỉ lệ nhấp chuột và tính hồi quy của CTR, giả sử các campaign đều gửi với số email như nhau và ở trong bộ dữ liệu dùng để huấn luyện đã có sẵn CTR thật, ta cần sử dụng mô hình ML để ước lượng và dự báo CTR. Vì thế, ta thực hiện ước lượng và dự báo trên nhiều mô hình học có giám sát (supervised learning) như **k-NN**, **Random Forest**, **SVM** và **Linear Regression** theo hướng hồi quy.

2.5.1. k-NN

k-NN (K Nearest Neighbors) là một mô hình hoạt động bằng cách dự đoán một giá trị target của item mới dựa trên giá trị của “láng giềng” gần nhất. Mô hình k-NN có thể được sử dụng với bài toán hồi quy bằng cách ước lượng trung bình của giá trị target của k “láng giềng”. Trong đề tài sẽ sử dụng hàm khoảng cách Euclid giữa 2 điểm dữ liệu x và y trong không gian đặc trưng R^p được Han et al. (2012) phát biểu theo công thức:

$$d(x, y) = \left(\sum_{i=1}^n (x_i - y_i)^2 \right)^{\frac{1}{2}}$$

Với bài toán hồi quy, cho giá trị của điểm cần dự báo là $\hat{x}$, giá trị của điểm i trong tập huấn luyện là $x^{(i)}$, ta sẽ có:

$$d(\hat{x}, x^{(i)}) = \left(\sum_{i=1}^n (\hat{x}_i - x_i^{(i)})^2 \right)^{\frac{1}{2}}$$

2.5.2. Random Forest

Random Forest có thể hiểu là tập hợp của hai hay nhiều **cây quyết định (Decision Tree)**. Với bài toán ước lượng/ dự báo hồi quy, Breiman (2001) định nghĩa Random Forest được hình thành bởi nhiều cây phụ thuộc vào một vector ngẫu nhiên (θ), mà bộ dự đoán dạng cây $h(x, \theta)$ trả về các giá trị số thay vì các nhãn phân loại. Bộ dữ liệu phù hợp cho việc huấn luyện bằng Random Forest với CTR là biến liên tục.

2.5.3. SVM và SVR

SVM (Support Vector Machines) là mô hình học máy có giám sát (supervised learning), có thể thực hiện các bài toán hồi quy và phân loại, đặc biệt là phân loại nhị phân với việc tìm một siêu phẳng (hyperplane) có margin lớn nhất. Với bài toán hồi quy, **SVR (Support Vector Regressor)** dạng phi tuyến ta sẽ sử dụng Kernel SVR để dự báo chính xác nhất.

Công thức Kernel SVR được Awad and Khanna (2015) phát biểu như sau:

$$\min \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i + \xi_i^*$$

$$s. t \ f(x) = \sum_{i=1}^n (\alpha_i^* - \alpha_i) k(x_i, x) \ , k(x_i, x) = \varphi(x_i) \varphi(x)$$

Trong đó:

Hàm mục tiêu bao gồm w là một vector trọng số, $\frac{1}{2} \|w\|^2$ là thành phần chuẩn hóa độ lớn của vector trọng số, giúp SVR có thể tìm được siêu phẳng có biên nhỏ nhất (độ phức tạp thấp nhất), C là một hệ số phạt giúp điều chỉnh độ phức tạp của mô hình và sai số huấn luyện, giá trị C càng lớn thì mô hình có xu hướng khớp càng nhiều nhất có thể, gây ra hiện tượng overfitting.

Hàm ràng buộc bao gồm $\xi_i + \xi_i^*$ là biến slack (độ lệch) thể hiện sai số vượt quá ε , $\alpha_i^* - \alpha_i$ là nhân tử Lagrange trong bài toán đối ngẫu, Kernel (hàm hạt nhân) $k(x_i, x)$ đo độ tương đồng của dữ liệu x và x_i , $\varphi(x_i) \varphi(x)$ là các ánh xạ dữ liệu của x và x_i .

2.5.4. Linear Regression

Hồi quy tuyến tính được sử dụng để mô hình hóa mối quan hệ tuyến tính của biến mục tiêu (target) với một hay nhiều biến độc lập (features). Công thức về hồi quy tuyến tính đã được giải thích ở trên. Đối với học máy, ta có thể hiểu đơn giản hơn với công thức hồi quy tuyến tính cơ bản của Han et al. (2012) khi giả sử y là một biến phản hồi ngẫu nhiên thể hiện dưới dạng một phương trình tuyến tính của một biến ngẫu nhiên x :

$$y = wx + b$$

Với w, b lần lượt là hệ số góc và hệ số chặn. Đây là một phương trình đặc trưng và phổ biến của mô hình này.

2.5.5. Các chỉ số đánh giá mô hình

Để có thể biết được hiệu quả của ước lượng/ dự báo CTR, các chỉ số cơ bản trong đánh giá mô hình như RMSE (Root Mean Squared Error), MSE (Mean Squared Error), MAE (Mean Absolute Error) được sử dụng để so sánh giá trị huấn luyện thứ i x_i và giá trị được ước lượng/ dự báo thứ i $\hat{x}_i$ và đánh giá tính ưu việt của 4 mô hình trên..

$$MSE = \frac{\sum_{i=1}^n (x_i - \hat{x}_i)^2}{n}, RMSE = \sqrt{\frac{\sum_{i=1}^n (x_i - \hat{x}_i)^2}{n}}, MAE = \frac{1}{2} \sum_{i=1}^n |x_i - \hat{x}_i|$$

Chỉ số MAPE (Mean Absolute Percentage Error) sẽ không được áp dụng trong đề tài vì biến target có trường giá trị rất nhỏ $[0,1]$ khiến cho giá trị MAPE vô cùng lớn, không thể tính toán được.

2.5.6. Kiểm định chéo (Cross-validation)

Cross-validation (Kiểm định chéo) là một phương pháp được sử dụng để kiểm tra độ tổng quát hóa của mô hình học máy trên dữ liệu chưa biết.

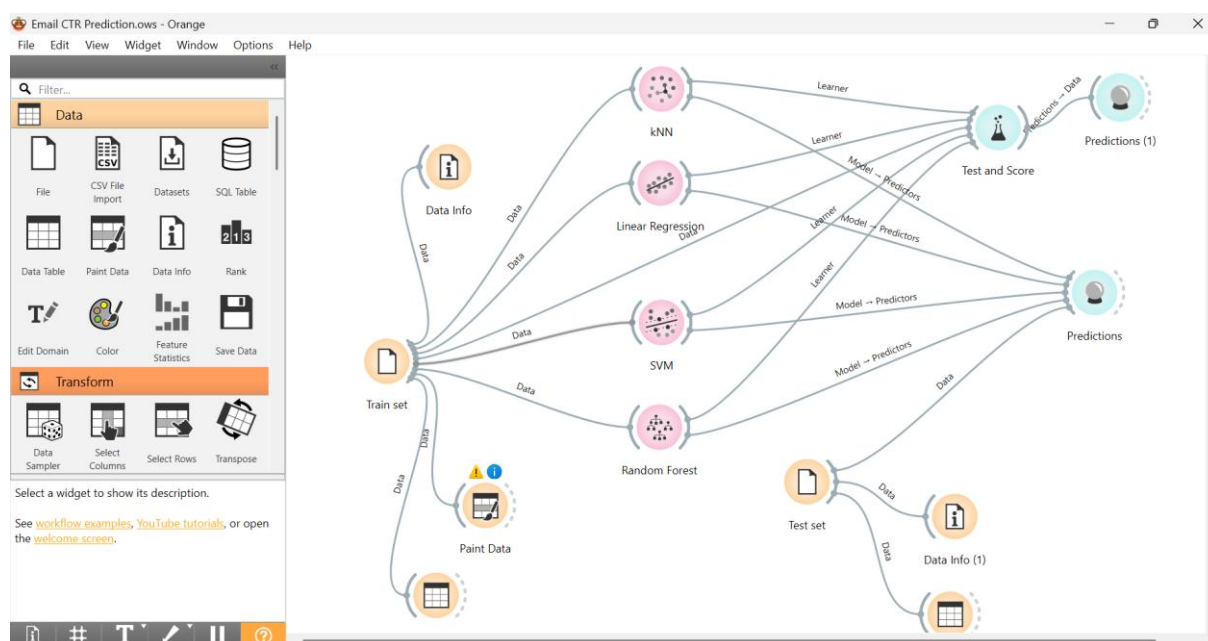
Theo Kohavi (1995), ta chia bộ dữ liệu thành k phần có kích thước gần bằng nhau (k -folds), mô hình sẽ được huấn luyện trên $k - 1$ phần và kiểm tra hoặc ước lượng trên phần còn lại, quá trình này sẽ thực hiện k lần.

Các dạng cross-validation phổ biến bao gồm k -fold (chia dữ liệu thành k phần), Stratified (dùng khi dữ liệu có phân phối không đồng đều của biến mục tiêu), Leave-one-out (mỗi mẫu được dùng làm tập kiểm tra một lần)

Đề tài sẽ sử dụng phương pháp này để đánh giá các mô hình ML hồi quy với phương pháp cross-validation k -folds = 5 và thực hiện chia tập dữ liệu theo thuộc tính times_of_day với nhiều phương tiện khác nhau nhằm có thêm nhiều góc nhìn của bài toán đánh giá/ dự báo.

2.5.7. Ước lượng/ Dự báo bằng Orange

Orange là một công cụ (toolbox) dành cho việc khai phá dữ liệu (data mining) được phát triển bằng ngôn ngữ Python. Các nhánh chính của công cụ bao gồm tiền xử lý dữ liệu, thực hiện bài toán phân lớp, hồi quy, phân cụm, đánh giá mô hình,... Đặc biệt, Orange còn hỗ trợ nền tảng lập trình trực quan (visual programming) với thành phần giao diện dành cho người dùng phục vụ cho việc tương tác và trực quan hóa dữ liệu (Demšar et al., 2013).



Hình 2.1. Giao diện Orange

Trong đó, các mô hình học máy đã được đề cập trước đó hiện tại đã có sẵn trên Orange. Ta sẽ sử dụng công cụ trực quan hóa Orange để huấn luyện và đánh giá hiệu quả của 4 mô hình SVM, Linear Regression, Random Forest, k-NN dựa trên đầu vào là bộ dữ liệu được cung cấp sẵn trên Kaggle trong bài toán ước lượng/ dự báo mà không cần lập trình phức tạp. Ngoài Orange, ta có thể kết hợp thêm với một thư viện Python khác để đa dạng phương thức dự báo/ ước lượng.

2.5.8. Ước lượng/ Dự báo bằng thư viện Python SciKit-Learn

Thư viện SciKit-Learn (Pedregosa et al. 2011) là một module học máy (Machine Learning) mã nguồn mở trong Python, cho phép giải các bài toán học có giám sát

(supervised learning) và học không giám sát (unsupervised learning). Mục tiêu của Scikit-Learn là đưa các phương pháp ML đến gần hơn với những người dùng không chuyên.

Scikit-Learn được thiết kế thành 2 phần chính: estimator và transformer. Estimator dùng phương pháp `fit()` để huấn luyện mô hình. Transformer bao gồm 2 phương thức `predict()` và `transform()`. Ngoài ra, thư viện còn cung cấp các hàm đánh giá các mô hình học máy như `score()`, `cross_val_score()`. Pipeline cho phép kết hợp nhiều bước (tiền xử lý, đánh giá, dự báo) thành một chuỗi với nhau, dễ dàng cho việc phân tích dữ liệu. (Pedregosa et al. 2011).

Với đề tài này, ta sẽ tận dụng thư viện Scikit-learn để thực hiện đánh giá 4 mô hình học máy kể trên bằng các chỉ số truyền thống rồi ước lượng/ dự báo CTR. Tất cả các công việc này sẽ được thực thi trên môi trường Python để đồng bộ với các bài toán sau và là đầu vào cho bài toán tối ưu hóa trong khung SPO (Smart Predict-then-Optimize).

2.6. Các vấn đề và phương pháp tối ưu hóa

Sau khi đã thực hiện bài toán ước lượng/ dự báo tỉ lệ nhấp chuột (CTR) thì còn rất nhiều yếu tố khách quan về mặt ràng buộc ngân sách, chi phí bỏ ra và việc hạn mức khi mà gửi email trong các khung thời gian nhất định. Do đó ta cần một tập chiến dịch được tối ưu, đáp ứng được các yếu tố đó. Bài toán tối ưu hóa được sử dụng để lựa chọn tập chiến dịch có hiệu quả cao nhất nhờ vào tập dự báo CTR của các mô hình học máy. Trong đề tài sẽ quan tâm đến 3 vấn đề: tối ưu hóa danh định, MILP, và thuật toán tham lam (dạng cơ bản của MILP).

2.6.1. Tối ưu hóa danh định (Nominal Optimization Problem)

Bài toán tối ưu danh định (Nominal Optimization Problem) được Elmachroub & Grigas (2017) phát biểu như công thức:

$$P(c) : z^*(c) := \min c^T w \text{ s.t. } w \in S$$

Với S là tập ràng buộc khả thi (feasible set), $c \in R^d$: là vector tham số của bài toán (problem data), w là một vector biến quyết định, $w \in \{0,1\}$, $c^T w$ là hàm mục tiêu tuyến tính thể hiện giá trị mục tiêu, $z^*(c)$ là một vector quyết định đã được tối ưu.

Khi giải trường hợp cụ thể mà tham số c chưa biết, giá trị dự báo của c sẽ được sử dụng thay thế. Tác giả bài báo giả định rằng khi ta có quyết định từ bài toán tối ưu hóa (oracle), được ký hiệu là $w^*(\hat{c})$, sẽ trả về $P(c)$ tương ứng với bất kỳ vector chi phí đầu vào. Nếu công thức ở trên thể hiện tuyến tính hoặc tối ưu hóa hỗn hợp số nguyên (mixed-integer optimization), thì ta cần một thư viện giải tối ưu hóa (solver) để tìm nghiệm $w^*(\hat{c})$

Như vậy đầu vào của bài toán tối ưu là bài toán dự báo từ các mô hình học máy. Các tác giả bài báo chấp nhận việc áp dụng thư viện tối ưu hóa trong vấn đề, vì thế trong đề tài này, thư viện MILP (Mixed-Integer Linear Optimization) được sử dụng, phù hợp với vấn đề tối ưu hóa được đặt ra, thư viện sẽ được giới thiệu ở phần sau.

2.6.2. MILP (Mixed-Integer Linear Programming)

MILP là một dạng bài toán tối ưu hóa tuyến tính với biến quyết định, phụ thuộc vào các biến ràng buộc phải là số nguyên (integer), các biến còn lại có thể là số thực.

Vì MILP có nhiều cách diễn giải khác nhau, nên ta sẽ lấy 1 công thức tiêu biểu được Gurobi Optimization (2025) triển khai:

$$\begin{aligned} & \min c^T x \\ & s.t \ b_l \leq Ax \leq b_u, l \leq x \leq u, x_i \in \mathbb{Z}, i \in X_i \end{aligned}$$

Với x là vector biến quyết định, A là ma trận, b_l, b_u, l, u là các vector giới hạn (bound), l là giới hạn dưới, u là giới hạn trên, X_i là một tập vị trí của biến quyết định.

2.6.3. Thuật toán “tham lam” Knapsack (Greedy Knapsack)

Trong quá trình giải MILP, giới hạn về mặt máy tính vật lý cũng như một số dự báo từ mô hình học máy có thể bị sai số quá nhiều khiến cho hiệu suất giải bài toán tối ưu bị

ảnh hưởng. Thuật toán Greedy Knapsack được sử dụng như một phương pháp thay thế và là một trường hợp đặc biệt của MILP trong trường hợp mô hình dự báo không khả thi cho việc giải MILP.

Thuật toán Greedy Knapsack được đề cập đầu tiên trong bài báo của Dantzig (1954) với bài toán 0-1 Knapsack) mô tả về một người leo núi muốn mang các vật phẩm đồ dùng khác nhau giả sử a_j là trọng lượng của vật phẩm j , b_j là giá trị (mức độ hữu ích) của vật phẩm j , x là một biến quyết định (lấy hoặc không lấy) thì hàm mục tiêu sẽ là:

$$z = \sum_{j=1}^n b_j x_j \text{ s.t } x_j \in \{0,1\}$$

Với ràng buộc không thể vượt quá sức chứa W :

$$\sum_{j=1}^n a_j x_j \leq W$$

Thuật toán này được coi là đơn giản hơn về mặt tính toán hơn so với MILP nhưng chỉ có giá trị so sánh khi đánh giá các mô hình nên khi ta chỉ sử dụng phương pháp Knapsack khi mà MILP giải bài toán vượt quá thời gian cho phép đối với một mô hình.

2.6.4. Thư viện Python dành cho tối ưu hóa SciPy.optimize.milp

Để phục vụ cho vấn đề tối ưu hóa, thư viện Python SciPy.optimize.milp (*Milp* — SciPy v1.16.2 Manual, n.d.) ra đời để giải trên nền tảng bài toán tối ưu hóa hỗn hợp số nguyên tuyến tính MILP (Mixed Integer Linear Programming) được đề cập ở trên.

Hàm **milp** trong Python được viết như sau:

```
milp(c, *, integrality=None, bounds=None, constraints=None, options=None)
```

Trong thư viện này có sự tham gia của các tham số:

- **c**: giá trị của hàm mục tiêu cần cực tiểu. Tham số c cần được đổi sang kiểu mảng số thực trước khi giải bài toán.
- **integrality**: xác định được ràng buộc toàn vẹn trên từng biến quyết định (decision variable) mang các giá trị:
 - 0: biến liên tục; không có ràng buộc toàn vẹn
 - 1: biến số nguyên; biến quyết định cần mang giá trị nguyên trong vùng giới hạn.
 - 2: biến bán liên tục (semi-continuous variable); biến quyết định phải nằm trong vùng giới hạn hoặc nhận giá trị 0
 - 3: biến bán số nguyên (semi-integer variable); biến quyết định cần mang giá trị nguyên trong vùng giới hạn.
- **bounds**: đường biên hay là giới hạn trên biến quyết định. Giới hạn trên và giới hạn dưới cần đổi sang mảng số thực trước khi giải bài toán. Nếu không xác định đường biên, các biến quyết định mang ràng buộc không âm.
- **constraints**: là một class mang ràng buộc tuyến tính của bài toán tối ưu. Ràng buộc tuyến tính tồn tại dưới dạng $b_l \leq Ax \leq b_u$ và được viết trong một class:


```
class LinearConstraint(A, lb=-inf, ub=inf, keep_feasible=False)
```

Có các ràng buộc theo khoảng, ràng buộc hai bên và ràng buộc một bên tùy vào việc đặt biên **lb** và **ub**. Ta sử dụng **np.inf** để triển khai ràng buộc một bên. Mặc định là ràng buộc không giới hạn **lb=-np.inf** và **ub=np.inf**.
- **options**: tùy chọn cho việc giải bài toán. Để đảm bảo được hiệu suất giải bài toán tối ưu với cấu hình vật lý có hạn, tùy chọn **time_limit** được sử dụng. **time_limit** là thời gian cho phép để giải bài toán tối ưu (được tính bằng giây). Mặc định không có giới hạn thời gian.

2.7. Khung SPO (Smart Predict-then-Optimize), hàm mất mát (SPO Loss) và mô hình hóa bài toán SPO trong Email Marketing.

Sau khi đã khái quát về vấn đề ước lượng/ dự báo và vấn đề tối ưu hóa, ta đưa 2 vấn đề này vào 1 vấn đề chung là Dự báo rồi tối ưu hóa (Predict-then-Optimize). Trong quá trình nghiên cứu về vấn đề dự báo rồi tối ưu hóa, khung lý thuyết SPO (Smart Predict-then-Optimize) được coi là phù hợp nhất. Dựa trên bài báo của Elmachetoub & Grigas (2017) về vấn đề dự báo rồi tối ưu một cách “thông minh”, chất lượng ước lượng không chỉ được đánh giá qua sai số (MSE, MAE, RMSE) mà còn qua một hàm mất mát SPO +/- *SPO Loss (Smart Predict-then-optimize Loss)*. *SPO Loss* là một chi phí tổn thất khi đưa ra một quyết định thấp hơn mức tối ưu do việc dự báo sai lệch. Đó là cốt lõi của khung lý thuyết SPO.

Trong bối cảnh bài toán ta có thể hiểu *SPO Loss* tức là phần chênh lệch giá trị kinh tế giữa quyết định tối ưu thật (biết CTR thật) và quyết định dựa trên CTR được ước lượng bởi mô hình.

Bài báo đi sâu vào nguyên lý hoạt động của hàm mất mát SPO+ là hàm xấp xỉ lồi (convex surrogate) của hàm mất mát SPO để phục vụ cho việc huấn luyện học máy cho việc ra quyết định. Ngoài cách tiếp cận truyền thống là tối thiểu hóa (minimization) các chỉ số đánh giá mô hình (MSE, RMSE, MAE), SPO+ cho phép mô hình học theo hướng giảm thiểu chi phí quyết định sai.

Do giới hạn về mặt học thuật vì việc huấn luyện toàn bộ bằng SPO+ vượt quá khả năng của sinh viên, phần thực nghiệm chỉ tập trung vào việc tính toán, đo lường chỉ số SPO Loss.

Từ khung lý thuyết đó ta sẽ xét đến bài toán ước lượng/ dự báo sau đó tối ưu hóa tập chiến dịch Email Marketing và được tích hợp với hàm mất mát SPO.

2.7.1. Thiết kế mô hình tích hợp SPO (ước lượng/ dự báo rồi tối ưu hóa) trong Email Marketing và đánh giá bằng SPO Loss.

Input: bao gồm train set (dùng cho việc huấn luyện mô hình và ước lượng) và test set (dùng cho việc dự báo) trong bộ dữ liệu Email CTR Prediction.

Output: các quyết định tối ưu là các chiến dịch được chọn.

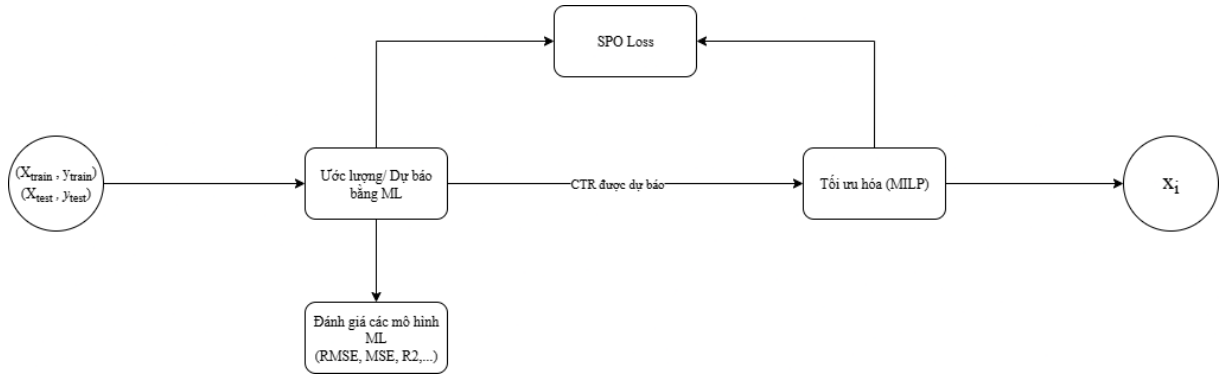
Các mô hình học máy được sử dụng trong việc ước lượng bao gồm Linear Regression, Random Forest, k-NN và SVM.

Phương pháp huấn luyện, đánh giá các mô hình cho dự báo: áp dụng cross-validation (chia theo folds, chia theo feature times_of_day); các chỉ số như RMSE, R2, MSE, MAE.

Sau khi dự báo CTR, dữ liệu được dự báo và các chỉ số đánh giá sẽ được tích hợp vào bài toán tối ưu bằng MILP và sau đó được đánh giá thêm bằng chỉ số SPO Loss.

Do test set không có chỉ số CTR thật nên ta sẽ không thực hiện tính toán SPO Loss trên tập này mà sử dụng cross-validation trên train set để tính giá trị tổn thất khi dự báo sai lệch so với giá trị thực tế.

Ta sẽ giải thích rõ hơn về 2 tầng làm việc của khung SPO.



Hình 2.2. Sơ đồ luồng làm việc của mô hình tích hợp SPO và đánh giá bằng SPO Loss

2.7.2. Tầng ước lượng/ dự báo (Estimate/ Predict)

Vì biến click_rate là một tỷ lệ thực nằm trong khoảng $[0,1]$ nên ta có thể thực hiện hồi quy (Regression), trong đó mục tiêu là áp dụng các mô hình học máy bao gồm k-NN Regression, Random Forest, Linear Regression, SVM để ước lượng CTR cho từng chiến dịch email dựa trên các đặc trưng đầu vào.

Hàm ước lượng được ký hiệu:

$$\hat{p}_i = f(features_i)$$

Trong đó:

Biến mục tiêu là $\hat{p}_i$: CTR được ước lượng/ dự báo.

Các biến thuộc tính (features) là những đặc trưng trong nội dung email (*subject_len, body_len, mean_paragraph_len, no_of_CTA, mean_CTA_len, is_discount*), ngữ cảnh gửi (*day_of_week, is_weekend, times_of_day*), cũng như đặc trưng sản phẩm/danh mục (*category, product*).

2.7.3. Tầng tối ưu (Optimize)

Sau khi đã thực hiện ước lượng/ dự báo CTR từ các mô hình học máy trên, doanh nghiệp cần biết liệu chỉ số này có thể giúp tìm ra những chiến dịch có giá trị kinh tế tốt nhất, vì

thế ta có thể đưa CTR dự báo vào vấn đề tối ưu, tức là liệu ta có thể chọn được chiến dịch đó được không.

Bài toán tối ưu sẽ là:

$$F(x): f(x) := f(x_i) \text{ s.t } g(x) \leq b, x \in \{0,1\}^n$$

Trong đó:

x là *biến quyết định*, giá trị của $x_i \in \{0,1\}$, kết quả là một dãy số nhị phân (VD: với số campaign được lấy là 8, $x = (01001011)$, với $x_1 = 0$ (*chiến dịch 1 không chọn*), $x_2 = 1$ (*chiến dịch 2 chọn*), $x_3 = 0, \dots, x_8 = 1$).

Như vậy chiến dịch 2, 4, 7, 8 được chọn)

$f(x)$ là một **hàm mục tiêu**.

$g(x)$ là một ràng buộc phụ thuộc vào tham số ràng buộc b .

2.7.4. Hàm mục tiêu và các ràng buộc phục vụ cho bài toán tối ưu

Ta có *hàm mục tiêu khi ước lượng/ dự báo*:

$$f(x_i) = \sum_{i=1}^n \widehat{p}_i * V_i * x_i$$

$\widehat{p}_i$ là giá trị tỉ lệ nhấp chuột (CTR) được ước lượng ở mỗi campaign

x_i là một biến quyết định tại giá trị i . Trong bối cảnh bài toán thì $x_i \in \{0,1\}$. VD: ở chiến dịch 2 được chọn $\Rightarrow x_2 = 1$

V_i là doanh thu có được với mỗi cú nhấp chuột, VD: 1 cú nhấp chuột có thể tạo ra được doanh thu 1000 cho 1 chiến dịch

Và hàm mục tiêu thực tế:

$$f(x_i) = \sum_{i=1}^n p_i * V_i * x_i$$

$\hat{p}_i$ là giá trị tỉ lệ nhấp chuột (CTR) thực tế

x_i là một biến quyết định tại giá trị i . Trong bối cảnh bài toán thì $x_i \in \{0,1\}$. VD: ở chiến dịch 2 được chọn $\Rightarrow x_2 = 1$

V_i là doanh thu có được với mỗi cú nhấp chuột, VD: 1 cú nhấp chuột có thể tạo ra được doanh thu 1000 cho 1 chiến dịch

Dựa trên CTR được ước lượng, doanh nghiệp cần giải bài toán tối ưu hóa lợi ích kinh tế bằng cách chọn **tập campaign có giá trị kinh tế cao nhất**, tuân thủ các ràng buộc thực tế như: quota số campaign theo khung giờ, tổng ngân sách, chi phí có thể bỏ ra.

Ràng buộc $g(x) \leq b$ tổng quát sẽ là phương trình ma trận:

$$Ax \leq B.$$

Trong đó

A chứa hệ số của các điều kiện ràng buộc

B là hạn mức (quota, ngân sách,...).

Các ràng buộc phụ thuộc:

- **Phải đúng theo khung giờ gửi cũng như đặt quota (hạn mức tối đa) theo slot:**

Ví dụ: Có 3 khung giờ để gửi email bao gồm morning, noon và evening tương ứng với Q_M, Q_N, Q_E là các chiến dịch được gửi:

$$\sum_{i \in \text{Morning}} x_i = Q_M, \sum_{i \in \text{Noon}} x_i = Q_N, \sum_{i \in \text{Evening}} x_i = Q_E$$

Ta có thể suy ra tổng số chiến dịch cần lấy:

$$Q = Q_M + Q_N + Q_E$$

Với việc từng quota cũng bị chặn bởi số lượng ứng viên sẵn có:

$$0 \leq Q_M \leq |M|, 0 \leq Q_N \leq |N|, 0 \leq Q_E \leq |E|$$

VD: Buổi sáng có 3 chiến dịch được gửi: $\sum_{i \in \text{Morning}} x_i = Q_M = 3$

Buổi trưa có 4 chiến dịch được gửi $\sum_{i \in \text{Noon}} x_i = Q_N = 4$

Buổi tối có 5 chiến dịch được gửi $\sum_{i \in \text{Evening}} x_i = Q_E = 5$

Vậy tổng số chiến dịch cần gửi là $Q = Q_M + Q_N + Q_E = 3 + 4 + 5 = 12$, tức tổng hạn mức ta có thể gửi theo slot là 12 chiến dịch

- **Chi phí ngân sách** (suy ra từ cấu trúc email/ nguồn lực) là c_i càng thấp càng tốt:

$$\sum_i c_i x_i \leq C$$

Trong đó C là giới hạn ngân sách chi tối đa cho các chiến dịch. Nếu 1 campaign có body dài hơn, subject dài hơn thì sẽ tốn nhiều nguồn lực (designer, content writer, thời gian gửi), trọng số do doanh nghiệp đặt ra và chỉ mang giá trị ràng buộc.

VD: Nếu 1 campaign A có độ dài tiêu đề email là 60, độ dài thân email là 240, trọng số chi phí mà doanh nghiệp có thể bỏ ra là 0.3:0.7 thì chi phí có thể bỏ ra là $60 * 0.3 + 240 * 0.7 = 186$

2.7.5. Hàm mất mát SPO Loss

Hàm mất mát SPO Loss được Elmachetoub & Grigas (2017) định nghĩa như sau:

Giả sử cho một vector chi phí trong thực tế là $c \in R^d$,

Quyết định tối ưu $z^*(c)$ sẽ được xác định bởi tối ưu hóa danh định đã đề cập trước đó. Sau đó 2 vector chi phí từ 2 quyết định sẽ được tính toán để cho ra chỉ số SPO Loss với vector chi phí được dự báo là $\hat{c}$, từ đó cho ra quyết định tối ưu $w^*(\hat{c})$ dựa trên việc ước lượng:

SPO loss = Chi phí của quyết định tối ưu sau khi dự báo/ ước lượng - Chi phí của quyết định tối ưu trong thực tế

$$l_{SPO}^{w^*}(\hat{c}, c) := c^\top w^*(\hat{c}) - z^*(c)$$

Ta sẽ dùng hàm mất mát SPO vào vấn đề được đặt ra trong đề tài này nhằm thực hiện so sánh độ hiệu quả hậu tối ưu từ dự báo của các mô hình học máy.

Định nghĩa trong bài báo khẳng định rằng SPO Loss thể hiện dưới dạng cực tiểu chi phí (minimization), vì thế khi áp dụng được đặt ra với mỗi campaign i , CTR được dự báo trước là $\hat{p}_i$, CTR thực tế là p_i và giá trị kinh tế từ chiến dịch mang lại là V_i , ta cần phải viết vector chi phí thực dưới dạng:

$$c_i = -p_i V_i, \hat{c}_i = -\hat{p}_i V_i.$$

Ta giải được bài toán tối ưu với hàm mục tiêu $f(x_i) = \sum_{i=1}^n \widehat{p}_i * V_i * x_i$, thì sẽ có chi phí của quyết định dựa trên dự báo là $s * (\widehat{c}_i)$, tương tự ta cũng có chi phí dựa trên thực tế (oracle) $r * (c_i)$, thì SPO Loss sẽ là

$$l_{SPO}^s(\widehat{c}_i, c_i) := c_i^\top s^*(\widehat{c}_i) - r^*(c_i)$$

Nếu $l_{SPO}^s(\widehat{c}_i, c_i) = 0$ thì quyết định tối ưu như trong thực tế.

Ví dụ: Giả sử có 2 campaign, với giá trị mỗi click $V = 1000$:

Campaign	CTR thật p_i	CTR dự báo $\widehat{p}_i$	Giá trị thực tế $p_i * V_i$	Giá trị từ việc dự báo $\widehat{p}_i * V_i$	Chi phí thực tế $-p_i * V_i$	Chi phí từ quyết định dự báo $-\widehat{p}_i * V_i$
1	0.05	0.10	50	100	-50	-100
2	0.12	0.08	120	80	-120	-80

- Bước 1: Giải bài toán tối ưu theo CTR thật (Oracle)

Giá trị thật: Campaign 1 = 50, Campaign 2 = 120.

Chọn campaign có giá trị lớn nhất $\rightarrow$ Campaign 2.

Giá trị thực tế = 120.

- Bước 2: Giải bài toán tối ưu theo CTR dự báo

Giá trị dự báo: Campaign 1 = 100, Campaign 2 = 80.

Chọn campaign có giá trị dự báo lớn nhất $\rightarrow$ Campaign 1.

Nếu đem quyết định này chấm bằng CTR thật $\rightarrow$ giá trị thực tế thu được = 50.

Tính SPO Loss:

Ở đây:

- Chi phí từ quyết định tối ưu qua dự báo nếu chọn Campaign 1 = -50
- Chi phí từ quyết định tối ưu thật (chọn Campaign 2) = -120

→ $SPO\ Loss = (-50) - (-120) = 70$ (chênh lệch thiệt hại).

Tức là mất 70 đơn vị giá trị do dự báo sai lệch.

Để có thể đánh giá được hiệu quả tối ưu dự báo của các mô hình học máy trên nhiều khía cạnh, ta sẽ tính trung bình SPO Loss trên mỗi chiến dịch, trung vị của SPO Loss, và độ sai số của SPO Loss.

2.7.6. Trực quan hóa SPO Loss bằng thư viện Python Matplotlib

Nhằm dễ hình dung hơn về so sánh độ hiệu quả của việc ra quyết định tối ưu dựa trên dự báo của các mô hình học máy, ta sử dụng một thư viện Python khác để trực quan hóa phân tích - Matplotlib. Theo Hunter (2007), Matplotlib là một gói thư viện trực quan hóa 2D được sử dụng trong Python cho việc phát triển ứng dụng, viết mã tương tác và tạo ra các hình ảnh có chất lượng mang tính xuất bản, tương thích với nhiều giao diện người dùng và hệ điều hành khác nhau. Thư viện hỗ trợ xây dựng nhiều biểu đồ như biểu đồ tròn (pie chart), biểu đồ đường (line plot), biểu đồ cột (bar chart), biểu đồ phân tán (scatter plot), ... nhằm phục vụ cho việc nghiên cứu khoa học.

Sau khi đã thực hiện tính toán SPO Loss trung bình, trung vị và sai số của SPO Loss, ta sẽ trực quan hóa kết quả đó thành một biểu đồ cột được xây dựng bằng thư viện Matplotlib. Biểu đồ này thể hiện giá trị tổn thất SPO Loss của từng mô hình, tìm xem mô hình nào có khả năng ra quyết định tối ưu dựa trên dự báo gần với quyết định tối ưu thật (oracle) nhất. Ta sẽ đánh giá 2 khía cạnh: tính ổn định và độ hiệu quả của các mô hình sau khi thực hiện cả 2 bài toán dự báo và bài toán tối ưu sau đó.

CHƯƠNG 3: DỮ LIỆU, CÁC PHƯƠNG PHÁP HIỆN THỰC BÀI TOÁN VÀ THỰC NGHIỆM

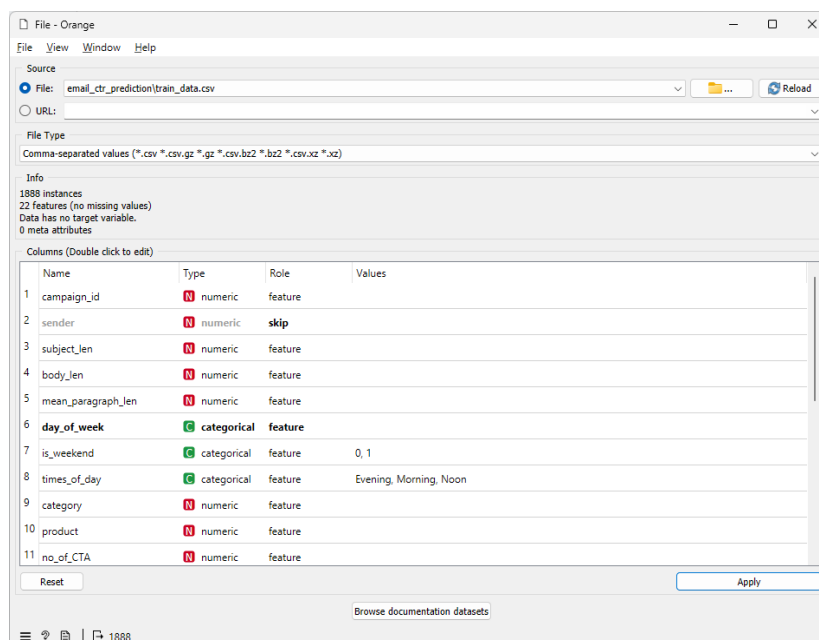
3.1. Giới thiệu bộ dữ liệu được sử dụng và các đặc trưng của dữ liệu

3.1.1. Tổng quan về dữ liệu

Bộ dữ liệu (Email CTR Prediction) được cung cấp trên Kaggle bao gồm các đặc trưng về nội dung email (subject_len, body_len, mean_paragraph_len, no_of_CTA, mean_CTA_len, is_discount), ngữ cảnh gửi (day_of_week, is_weekend, times_of_day), cũng như đặc trưng sản phẩm/danh mục (category, product). Biến mục tiêu (target) là **click_rate (CTR)** – tỷ lệ số lượt nhấp chuột vào liên kết/CTA trên tổng số email được gửi đi, luôn nằm trong khoảng $[0,1]$, **đã có sẵn trong dataset và ta gọi đó là CTR thật p_i** .

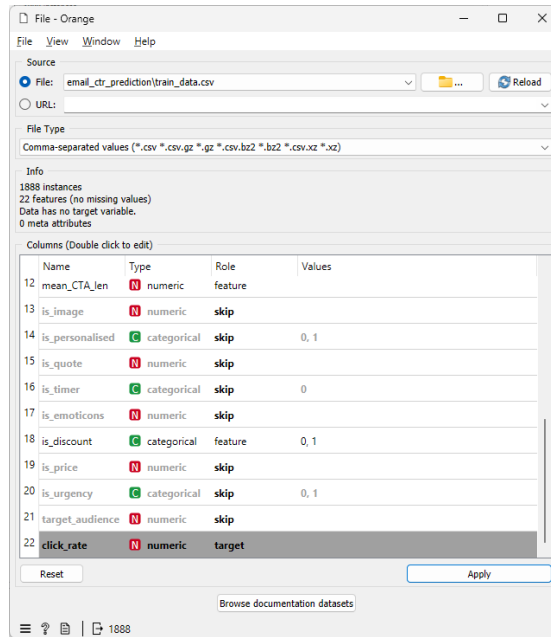
Trong bộ này có 2 file: 1 file train và 1 file test

Dữ liệu train bao gồm 1888 dòng (không có dữ liệu thiếu), bộ dữ liệu test bao gồm 762 dòng (không có click_rate).



Name	Type	Role	Values
1 campaign_id	numeric	feature	
2 sender	numeric	skip	
3 subject_len	numeric	feature	
4 body_len	numeric	feature	
5 mean_paragraph_len	numeric	feature	
6 day_of_week	categorical	feature	
7 is_weekend	categorical	feature	0, 1
8 times_of_day	categorical	feature	Evening, Morning, Noon
9 category	numeric	feature	
10 product	numeric	feature	
11 no_of_CTA	numeric	feature	

Hình 3.1. Bảng đặc trưng của tập dữ liệu train trong Orange



Hình 3.2. Bảng đặc trưng của dữ liệu test trong Orange

Các dòng dữ liệu của train set được thể hiện như ở dưới đây:

File Edit View Window Help

data

1888 instances (no missing data)
22 features
0 meta-attributes
Features: automatic
No meta-attributes (0 of 22)
100% numeric values
0 missing values
Order by relevance index
Selected rows

	click_rate	campaign_id	subject_len	body_len	text_paragraphs	day_of_week	is_weekend	time_of_day	category	product	is_CTA	mean_CTA_len	is_discount
1	0.000000	1	76	12410	28	1	Noon	6	28	3	28	0	
2	0.000000	2	54	2375	236	1	Noon	2	11	0	22	0	
3	0.000000	3	59	12851	16	1	Noon	2	11	3	23	0	
4	0.000000	4	74	11027	20	0	Evening	170	8	4	24	0	
5	0.000000	5	82	10011	27	1	Noon	6	28	3	21	0	
6	0.000000	6	74	2248	226	0	Evening	2	11	0	22	0	
7	0.000000	7	54	2570	236	0	Evening	2	11	0	22	0	
8	0.000000	8	62	12177	17	0	Noon	6	28	3	24	0	
9	0.000000	9	89	10050	14	0	Noon	151	9	4	28	0	
10	0.000000	10	89	11048	26	0	Evening	151	9	4	28	0	
11	0.000000	11	77	7552	36	0	Noon	6	28	3	25	0	
12	0.000000	12	78	10473	26	0	Noon	6	28	3	22	0	
13	0.000000	13	82	11957	23	0	Evening	151	9	4	27	0	
14	0.000000	14	86	14737	19	2	Evening	2	11	9	28	0	
15	0.000000	15	89	9467	22	0	Noon	6	28	3	17	0	
16	0.000000	16	78	10473	26	0	Noon	6	28	3	22	0	
17	0.000000	17	78	8525	20	0	Noon	6	28	3	40	0	
18	0.000000	18	83	9400	27	0	Noon	6	28	3	19	0	
19	0.000000	19	71	11334	36	0	Evening	151	9	4	31	0	
20	0.000000	20	71	11334	36	0	Evening	151	9	4	31	0	
21	0.000000	21	78	10473	26	0	Evening	6	28	3	22	0	
22	0.000000	22	57	7768	22	1	Noon	2	11	0	28	0	
23	0.000000	23	76	8883	48	0	Evening	151	9	4	28	0	
24	0.000000	24	88	8423	27	1	Noon	2	11	2	22	0	
25	0.000000	25	76	10473	26	0	Evening	6	28	3	22	0	
26	0.000000	26	88	8848	27	0	Noon	6	28	3	19	0	
27	0.000000	27	42	8852	48	0	Noon	6	10	3	15	0	
28	0.000000	28	82	8208	71	0	Noon	2	11	3	41	0	
29	0.000000	29	83	8848	22	0	Evening	6	28	3	31	0	
30	0.000000	30	82	10050	22	0	Evening	2	11	3	35	0	
31	0.000000	31	104	9882	48	0	Evening	151	9	4	34	0	
32	0.000000	32	88	10467	26	1	Noon	2	11	3	22	0	
33	0.000000	33	124	11420	27	0	Evening	151	9	4	47	0	
34	0.000000	34	88	11633	22	0	Noon	2	11	3	31	0	
35	0.000000	35	81	8787	20	0	Noon	6	28	3	31	0	
36	0.000000	36	76	11147	34	0	Evening	2	11	4	25	0	
37	0.000000	37	76	11147	34	0	Evening	2	11	4	25	0	
38	0.000000	38	82	12527	27	0	Evening	151	9	4	33	0	
39	0.000000	39	77	12527	27	0	Evening	151	9	4	33	0	
40	0.000000	40	81	8784	20	0	Noon	6	28	3	28	0	
41	0.000000	41	76	11147	34	0	Evening	2	11	3	25	0	
42	0.000000	42	88	9524	48	0	Evening	151	9	4	28	0	
43	0.000000	43	73	10272	22	0	Evening	151	9	4	23	0	
44	0.000000	44	83	11812	25	0	Evening	2	11	4	38	0	
45	0.000000	45	87	14633	22	0	Evening	151	9	4	23	0	
46	0.000000	46	73	11488	42	0	Noon	2	11	4	27	0	

Feature Selection Grid

☐ Drag Automatically

Hình 3.3. Thể hiện các dòng dữ liệu của train set

Các dòng dữ liệu của test set:

	campaign_id	sender	subject_len	body_len	non_paragraph_len	day_of_week	is_weekend	times_of_day	category	product
1	1889	3	61	12871	11	6	1	Noon	6	26
2	1890	3	54	2569	256	5	1	Morning	2	11
3	1891	3	88	1473	78	4	0	Evening	2	11
4	1892	3	88	1473	78	3	0	Evening	2	11
5	1893	3	78	9020	29	3	0	Noon	6	26
6	1894	3	99	14983	15	3	0	Noon	6	26
7	1895	3	104	9183	32	1	0	Evening	15	5
8	1896	3	86	14787	19	0	0	Evening	2	11
9	1897	3	64	12801	16	4	0	Evening	2	11
10	1898	3	63	11588	28	4	0	Noon	2	11
11	1899	3	63	11594	28	3	0	Evening	2	11
12	1900	3	81	9840	22	1	0	Evening	6	26
13	1901	3	76	9214	51	1	0	Evening	2	11
14	1902	3	80	11655	22	5	1	Noon	2	11
15	1903	3	80	15553	20	5	1	Noon	15	5
16	1904	3	84	15553	20	3	0	Evening	15	5
17	1905	3	84	15553	20	3	0	Evening	15	5
18	1906	3	81	10253	36	2	0	Evening	2	11
19	1907	3	65	11613	25	5	1	Noon	2	11
20	1908	3	67	8405	39	4	0	Noon	2	11
21	1909	3	65	11783	29	3	0	Evening	2	11
22	1910	3	68	11532	44	2	0	Noon	2	11
23	1911	3	104	9183	32	0	0	Evening	15	5

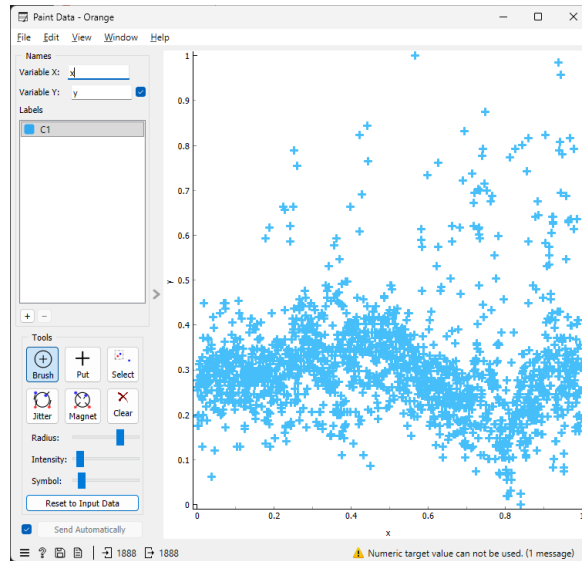
Hình 3.4. Thể hiện các dòng dữ liệu trong test set

	mean_CTA_len	is_image	is_personalised	is_quote	is_timer	is_emoticons	is_discount	is_price	is_urgency	target_audience
1	37	1	0	0	0	1	0	0	0	14
2	22	0	0	0	0	0	0	0	0	10
3	22	1	0	1	0	0	0	0	0	10
4	22	1	0	1	0	0	0	0	0	10
5	40	0	0	0	0	0	0	0	0	14
6	34	0	0	0	0	0	0	0	0	5
7	27	0	0	0	0	0	0	0	0	16
8	29	1	0	1	0	0	0	0	0	1
9	23	1	0	1	0	0	0	0	0	16
10	30	2	0	0	0	1	0	0	0	5
11	30	2	0	0	0	1	0	0	0	5
12	31	0	0	0	0	0	0	0	0	2
13	42	2	0	1	0	0	0	0	0	7
14	35	0	0	1	0	0	0	0	0	16
15	29	1	0	0	0	0	0	0	0	15
16	29	1	0	0	0	0	0	0	0	12
17	29	1	0	0	0	0	0	0	0	15
18	21	1	0	0	0	0	0	0	0	10
19	38	2	0	2	0	0	0	0	0	16
20	38	1	0	1	0	0	0	0	0	10
21	38	2	0	2	0	0	0	0	0	10
22	27	2	0	1	0	0	0	0	0	1
23	27	0	0	0	0	0	0	0	0	6

Hình 3.5. Thể hiện các dòng dữ liệu của test set - không có click_rate

3.1.2. Độ phân tán dữ liệu

Khi thực hiện trực quan hóa tính chất dữ liệu ta thấy dữ liệu phân tán rải rác - không nằm trên một đường thẳng rõ ràng, vì thế đây là *quan hệ phi tuyến*.



Hình 3.6. Biểu đồ thể hiện độ phân tán của dữ liệu

3.1.3. Thuộc tính (Features) được chọn

- **campaign_id**: mã số chiến dịch được đánh số bắt đầu từ 1, chỉ dùng để phân biệt các chiến dịch, không phải biến đầu vào mô hình.
- **subject_len**: độ dài của tiêu đề email, feature quan trọng dự báo CTR.
- **body_len**: độ dài thân email
- **mean_paragraph_len**: độ dài trung bình của đoạn văn trong nội dung email
- **day_of_week**: ngày trong tuần mà email được gửi đi
- **is_weekend**: đánh dấu email được gửi vào cuối tuần (1) hay ngày thường (0)
- **times_of_day**: khung giờ gửi email bao gồm morning, noon, evening
- **category**: số loại nội dung được nhắc đến trong email
- **product**: số lượng sản phẩm được quảng bá trong email
- **no_of_CTA**: số lượng nút kêu gọi hành động (Call-To-Action) trong email. Call-to-action là một chỉ dẫn cho khách hàng để tạo ra một phản ứng ngay lập tức. VD: sử dụng động từ như “gọi ngay bây giờ”, “tìm hiểu thêm” hoặc “ghé thăm

cửa hàng ngay hôm nay” sẽ kích thích khách truy cập thực hiện hành động sau khi đọc email.

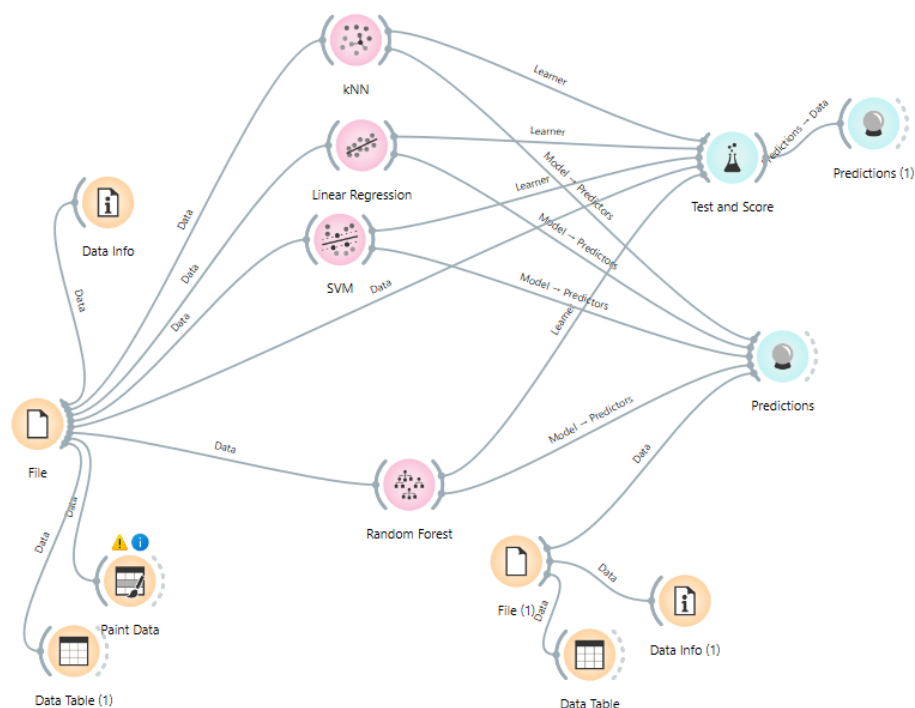
- **mean_CTA_len**: độ dài trung bình của 1 CTA
- **is_discount**: kiểm tra xem có giảm giá trong email hay không. Nếu có thì giá trị bằng 1, không thì giá trị bằng 0

3.1.4. Biến target (Biến mục tiêu)

click_rate: tỉ lệ nhấp chuột có tính liên tục chạy trong khoảng giá trị từ 0 đến 1.

3.2. Ước lượng/ dự báo

3.2.1. Ước lượng/ Dự báo bằng Orange



Hình 3.7. Workflow thực hiện dự báo trong Orange

B1: Xây dựng workflow Orange như trên để ước lượng với train set và dự báo trên test set với các mô hình học máy kNN, Linear Regression, SVM, và Random Forest.

B2: Thực hiện đánh giá các mô hình trên với việc chia thành 5 tập nhỏ hơn (Cross Validation 5 folds) và chia các tập theo features times_of_day để huấn luyện và ước lượng bằng các chỉ số như RMSE, MSE, MAE, R2.

B3: Ước lượng trên tập train set với 2 cách cross-validation khác nhau.

B4: Dự báo trên test set với các mô hình đã được huấn luyện.

3.2.2. Dự báo bằng Python với thư viện Scikit-learn

Với Python, thư viện **sklearn** được sử dụng cho việc huấn luyện các mô hình và dự báo, bao gồm cả tiền xử lý dữ liệu, đầu tiên ta cần thêm các thư viện Python (bao gồm cả sklearn) vào source code.

```
# -----Libs-----  
  
import numpy as np, pandas as pd  
from pathlib import Path  
  
# ML  
  
from sklearn.compose import ColumnTransformer  
from sklearn.preprocessing import OneHotEncoder,  
StandardScaler  
  
from sklearn.pipeline import Pipeline  
from sklearn.impute import SimpleImputer  
from sklearn.ensemble import RandomForestRegressor  
from sklearn.linear_model import LinearRegression  
from sklearn.svm import SVR  
from sklearn.neighbors import KNeighborsRegressor
```

```

from sklearn.metrics import mean_squared_error, r2_score,
mean_absolute_percentage_error, mean_absolute_error

from sklearn.model_selection import KFold

```

Ở trường hợp này, có thể thay đổi các thông số và các ràng buộc sẽ được sử dụng trong các bài toán như Cross Validation, nếu không có cột cost thì có thể tạo thêm một cột cost (ràng buộc chi phí có thể bỏ ra cho chiến dịch đó).

```

# ----- USER CONFIG AND INPUT -----

CV_FOLDS = 5

RF_TREES = 200

MLP_MAX_ITER = 200

SCIPY_TIME_LIMIT = 8 # seconds for MILP solver

#

print("\n=== Thiết lập tham số kinh tế & quota ===")

# Giá trị mặc định mỗi click

V_CONST = ask_float("Giá trị mỗi click (V_CONST), mặc định là", 1000.0)

# Nếu không có cột cost thì tự tạo proxy

# Lưu ý: có

MAKE_MONETARY_COST_IF_MISSING = ask_str(

    "Nếu thiếu cột chi phí thì tạo proxy từ body_len? (yes/no), mặc định", "yes"

```



```

).lower().startswith("y")

# Ngân sách: nhập số & chọn kiểu

BUDGET_MODE = ask_str("Ngân sách nhập kiểu (ratio / absolute / none), mặc định là", "ratio").lower()

if BUDGET_MODE == "ratio":

    BUDGET_IS_RATIO = True

    BUDGET_VALUE = ask_float("Nhập tỷ lệ ngân sách (mặc định 0.35 = 35%)", 0.35)

elif BUDGET_MODE == "absolute":

    BUDGET_IS_RATIO = False

    BUDGET_VALUE = ask_float("Nhập số ngân sách tuyệt đối, mặc định", 0.0)

else:

    BUDGET_IS_RATIO = False

    BUDGET_VALUE = 0.0

# Quotas cho times_of_day

QUOTAS = {}

for slot in ["morning", "noon", "evening"]:

    q = ask_str(f"Quota tối đa cho slot {slot} (trống = bỏ qua)", "")

    if q.strip() == "":

```

```

        QUOTAS[slot] = None

    else:

        try:

            QUOTAS[slot] = int(q)

        except:

            print(f" quota {slot} không hợp lệ, bỏ qua.")

            QUOTAS[slot] = None

#

# Optional generic linear constraints Ax <= B (on decision
vector x). Keep disabled by default.

USE_GENERIC_AXB = False

GEN_A = None

GEN_B = None

# SciPy MILP

try:

    SCIPY_OK = True

except Exception:

    print("Không thể thực hiện được bài toán tối ưu")

    exit(1)

```

Thực hiện huấn luyện trên tập train bằng cách chọn cột target và feature, thiết lập x và y.

```
# ----- Features/Target -----  
  
target_col = "click_rate"  
  
X_train_full = train_df.drop(columns=[target_col])  
  
y_train_full = train_df[target_col].astype(float)
```

Tiếp đó thực hiện việc tiền xử lý dữ liệu bằng cách xác định các biến phân loại và biến số học. Với các biến số học sẽ thực hiện điền dữ liệu thiếu bằng giá trị trung vị và chuẩn hóa dữ liệu về cùng một thang đo, với các biến phân loại sẽ thực hiện điền dữ liệu bằng giá trị xuất hiện nhiều nhất trong dữ liệu và chuyển biến phân loại thành biến nhị phân, với các giá trị mới ngoài tập huấn luyện thì sẽ bỏ qua.

```
# Identify numeric/categorical  
  
num_cols = X_train_full.select_dtypes(include=[np.number]).columns.tolist()  
  
cat_cols = [c for c in X_train_full.columns if c not in num_cols]  
  
numeric_tf = Pipeline([  
    ("imputer", SimpleImputer(strategy="median")),  
    ("scaler", StandardScaler(with_mean=False))  
)
```

```
categorical_tf = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("one", OneHotEncoder(handle_unknown="ignore"))
])

preprocess = ColumnTransformer(
    [
        ("num", numeric_tf, num_cols),
        ("cat", categorical_tf, cat_cols)
    ],
    remainder="drop"
)
```

Tạo một mảng liên quan đến các mô hình được sử dụng để huấn luyện bao gồm LinearRegression, Random Forest, SVM và k-NN.

```
models = {
    "LinearRegression": LinearRegression(),
    "RandomForest":
RandomForestRegressor(n_estimators=RF_TREES,
random_state=7, n_jobs=-1),
    "SVM(SVR)": SVR(kernel="rbf", C=10, gamma="scale"),
    "k-NN": KNeighborsRegressor(n_neighbors=5, n_jobs=-1)
}
```

Thực hiện Cross Validation trên tập dữ liệu train và được chia làm 5, cũng như thực hiện đánh giá mô hình dựa trên các chỉ số RMSE, MAE, R2.

```

# ----- Cross-Validation on TRAIN and Evaluate (OOF)
-----

cv_rows = []

kf      =      KFold(n_splits=CV_FOLDS,      shuffle=True,
random_state=42)

for name, est in models.items():

    oof = np.zeros(len(train_df), dtype=float)

    for tr, va in kf.split(X_train_full):

        X_tr,      X_va      =      X_train_full.iloc[tr],
X_train_full.iloc[va]

        y_tr,      y_va      =      y_train_full.iloc[tr],
y_train_full.iloc[va]

        pipe = Pipeline([("prep", preprocess), ("model",
est)])

        pipe.fit(X_tr, y_tr)

        pred  =  np.clip(pipe.predict(X_va).astype(float),
0.0, 1.0)

        oof[va] = pred

# -----Evaluate-----
-----

rmse = mean_squared_error(y_train_full, oof)

r2    = r2_score(y_train_full, oof)

mape_val = mean_absolute_percentage_error(y_train_full,
oof)

```

```

mae = mean_absolute_error(y_train_full, oof)

cv_rows.append({"Model": name, "RMSE (OOF)": rmse,
               "R2 (OOF)": r2, "MAPE (OOF)": mape_val, "MAE (OOF)": mae})

cv_df = pd.DataFrame(cv_rows).sort_values("RMSE (OOF)")

print("CV metrics (OOF on TRAIN)")

print(cv_df)

```

Sau khi đánh giá xong, huấn luyện toàn bộ train set, phục vụ cho việc dự báo CTR trên test set:

```

# ----- Fit on FULL TRAIN -> predict TEST -----

X_test = test_df.drop(columns=[target_col],
                           errors="ignore")

pipes, test_preds = {}, {}

for name, est in models.items():

    pipe = Pipeline([("prep", preprocess), ("model", est)])

    pipe.fit(X_train_full, y_train_full)

    yhat_test = np.clip(pipe.predict(X_test).astype(float),
                        0.0, 1.0)

    pipes[name] = pipe; test_preds[name] = yhat_test

```

3.3. Tối ưu hóa bằng thư viện Python MILP

Muốn giải được bài toán tối ưu danh định $P(c) : z^*(c) := \min \mathbf{c}^\top \mathbf{w}$, ta áp dụng thư viện tối ưu hóa của SciPy với phương pháp MILP (Mixed-Integer Linear Programming) cùng hướng dẫn đi kèm (*Milp — SciPy v1.16.2 Manual*, n.d.), với các ràng buộc đi kèm.

B1: Tạo trước class ràng buộc khung giờ gửi và quota theo slot, bằng cách tạo một class liên quan đến times_of_day.

```
def build_slot_quota_constraint(df,
slot_col="times_of_day", quotas=None):

    quotas = quotas or {}

    slots = list(quotas.keys())

    n = len(df)

    A, b = [], []

    for s in slots:

        q = quotas[s]

        if q is None:

            continue

        mask = (df[slot_col].astype(str).str.lower() ==
s.lower()).astype(float).values

        A.append(mask); b.append(float(q))

    if not A:

        return None, None

    return np.vstack(A), np.array(b)
```

B2: Tạo một class giải MILP nhằm thực hiện bài toán tối ưu với các điều kiện ràng buộc, bao gồm ràng buộc ngân sách, ràng buộc khung giờ gửi, quota theo slot, với đầu vào là vector chi phí đã có từ bài toán tối ưu trước đó để đưa vào hàm MILP.

```
# ----- Optimization -----
```

```

def solve_milp(df, cost_vec, budget=None, quotas=None,
A_extra=None, B_extra=None, time_limit=SCIPY_TIME_LIMIT):
    """
    Minimize cost_vec^T x subject to:
        sum(cost_monetary_i * x_i) <= budget (if budget is
not None)
        per-slot quotas (<=)
        A_extra x <= B_extra (if provided)
        x in {0,1}^n
    """
    n = len(df)
    c = np.asarray(cost_vec, float) # already the vector
cost to minimize
    integrality = np.ones(n, dtype=int)
    bounds = Bounds(lb=np.zeros(n), ub=np.ones(n))
    constraints = []
    # Budget on monetary cost (optional)
    if budget is not None and "cost" in df.columns:
        costs_money =
df["cost"].astype(float).values.reshape(1, -1)
        constraints.append(LinearConstraint(costs_money,
lb=-np.inf, ub=[float(budget)]))

```



```

# Slot quotas (optional)

if quotas is not None:
    A_slot, b_slot = build_slot_quota_constraint(df,
"times_of_day", quotas)

    if A_slot is not None:
        constraints.append(LinearConstraint(A_slot,
lb=-np.inf*np.ones(len(b_slot)), ub=b_slot))

# Generic Ax<=B (optional)

if A_extra is not None and B_extra is not None:
    A_extra = np.asarray(A_extra, float)
    B_extra = np.asarray(B_extra, float)

    if A_extra.ndim == 1:
        A_extra = A_extra.reshape(1, -1)

    if B_extra.ndim == 0:
        B_extra = B_extra.reshape(1, )

    constraints.append(LinearConstraint(A_extra, lb=-
np.inf*np.ones(A_extra.shape[0]), ub=B_extra))

# MILP call

if SCIPY_OK:
    try:

```

```

        res = milp(c=c, integrality=integrality,
bounds=bounds, constraints=constraints,

                    options={"time_limit": time_limit})

        if res.success and res.x is not None:

            return (res.x >= 0.5).astype(int), True

    except Exception:

        pass

```

Trường hợp không thể áp dụng được MILP do ước lượng/ dự báo sai số quá nhiều, ta sẽ thực hiện tính toán bằng thuật toán tham lam Knapsack đa chiều (MDKP) để phát hiện được tập chiến dịch mang lại lợi nhuận nhiều nhất.

```

# Fallback: simple density greedy on (-cost) as pseudo-
profit

pseudo_profit = -c # maximize -cost

density = pseudo_profit /
np.maximum(df["cost"].astype(float).values, 1e-6) if
("cost" in df.columns and budget is not None) else
pseudo_profit

order = np.argsort(-density)

x = np.zeros(n, dtype=int)

spent = 0.0

for j in order:

```

```

        if budget is not None and "cost" in df.columns:
            if spent + df["cost"].iloc[j] > budget:
                continue

        # check slot quotas greedily

        ok = True

        if quotas is not None and "times_of_day" in
df.columns:

            slot = str(df["times_of_day"].iloc[j]).lower()

            if slot in quotas and quotas[slot] is not None:

                # count current chosen in that slot

                cnt = (x == 1) &
(df["times_of_day"].astype(str).str.lower().values ==
slot)).sum()

                if cnt + 1 > int(quotas[slot]):

                    ok = False

        if ok:

            x[j] = 1

            if budget is not None and "cost" in df.columns:

                spent += float(df["cost"].iloc[j])

    return x, False

```

3.4. Đo lường chỉ số SPO Loss

Để có thể đo lường được chỉ số SPO Loss, ta cần giải bài toán tối ưu MILP 2 lần:

1. Tính được vector chi phí của CTR được dự báo từ đó cho ra quyết định dựa trên dự báo.
2. Tính được vector chi phí dựa trên CTR thật sau đó đưa ra quyết định oracle (quyết định dựa trên thực tế)

Sau khi tìm hiểu, thư viện pyepo (Python Environment for Predict-then-Optimize), được đề cập trong bài báo của (Tang & Khalil, 2022), tồn tại để phục vụ cho các bài toán liên quan đến Predict-then-Optimize. Tuy nhiên bài báo chỉ đề cập đến SPO+, không phù hợp với mục tiêu đồ án, cho nên ta sẽ thực hiện viết một class tính SPO Loss dựa vào công thức ở trên.

Vì tập test set không có click-rate thực cho nên không thể thực hiện tính SPO Loss, thay vào đó tập train set sẽ được dùng để tính toán để đánh giá độ hiệu quả của việc tối ưu qua dự báo.

Ta sẽ phải thực hiện 2 bài toán tối ưu ở CTR thật và CTR đã được dự báo ở tập train để có thể thực hiện tính được SPO Loss.

B1: Thực hiện lại bài toán dự báo để tính hàm chi phí (vector âm).

```
#-----SPO Loss-----
kf      =      KFold(n_splits=CV_FOLDS,      shuffle=True,
random_state=42)

rows_spo, rows_cv = [], []

for name, est in models.items():

    oof = np.zeros(len(train_df), dtype=float)

    for      fold_id,      (tr_idx,      va_idx)      in
enumerate(kf.split(X_train_full), start=1):
```

```

X_tr, X_va = X_train_full.iloc[tr_idx],
X_train_full.iloc[va_idx]

y_tr, y_va = y_train_full.iloc[tr_idx],
y_train_full.iloc[va_idx]

df_va =
train_df.iloc[va_idx].reset_index(drop=True)

pipe = Pipeline([("prep", preprocess), ("model",
est)])

pipe.fit(X_tr, y_tr)

p_pred = np.clip(pipe.predict(X_va).astype(float),
0.0, 1.0)

p_true = np.clip(y_va.astype(float).values, 0.0,
1.0)

oof[va_idx] = p_pred

V_vec =
df_va["value_per_click"].astype(float).values if
"value_per_click" in df_va.columns else np.full(len(df_va),
V_CONST)

c_true = -(p_true * V_vec)

c_pred = -(p_pred * V_vec)

if "cost" in df_va.columns and BUDGET_VALUE is not
None:

```

```

        total_cost_va = float(df_va["cost"].sum())

        budget_va = (BUDGET_VALUE * total_cost_va) if
BUDGET_IS_RATIO else float(BUDGET_VALUE)

    else:

        budget_va = None

```

B2: Thực hiện bài toán tối ưu 2 lần với tập train: 1 lần với CTR đã được dự báo và CTR thực tế đã có sẵn.

```

        x_true, used_true = solve_milp(df_va, c_true,
budget=budget_va,

quotas=QUOTAS,

A_extra=(GEN_A if USE_GENERIC_AXB else None),

B_extra=(GEN_B if USE_GENERIC_AXB else None),

time_limit=SCIPY_TIME_LIMIT)

        x_pred, used_pred = solve_milp(df_va, c_pred,
budget=budget_va,

quotas=QUOTAS,

A_extra=(GEN_A if USE_GENERIC_AXB else None),

```

```
B_extra=(GEN_B if USE_GENERIC_AXB else None),

time_limit=SCIPY_TIME_LIMIT)
```

B3: Tính SPO Loss bằng cách tính 2 vector chi phí với quyết định tối ưu của CTR thực và CTR dự báo.

```

true_obj = float(c_true @ x_true)

obj_pred_under_true = float(c_true @ x_pred)

spo = obj_pred_under_true - true_obj

rows_spo.append({"Model": name, "Fold": fold_id,
                  "UsedMILP(True)": used_true,
                  "UsedMILP(Pred)": used_pred,
                  "True_Objective": true_obj,
                  "Objective_under_true_of_x_pred":
obj_pred_under_true,
                  "SPO_Loss": spo})

rmse = mean_squared_error(y_train_full, oof)

r2    = r2_score(y_train_full, oof)
```

```

    mape_val = mean_absolute_percentage_error(y_train_full,
oof)

    rows_cv.append({"Model": name, "RMSE (OOF)": rmse,
"R2 (OOF)": r2, "MAPE (OOF)": mape_val})

spo_df = pd.DataFrame(rows_spo)
cv_df = pd.DataFrame(rows_cv).sort_values("RMSE (OOF)")

spo_summary = (spo_df.groupby("Model", as_index=False)
                .agg(Mean_SPO_Loss=("SPO_Loss", "mean"),
                    Median_SPO_Loss=("SPO_Loss", "median"),
                    Std_SPO_Loss=("SPO_Loss", "std"),
                    Folds_Done=("SPO_Loss", "count"))
                .sort_values("Mean_SPO_Loss"))

print("SPO Loss per fold (TRAIN)")
print(spo_df)

print("SPO Loss summary by model (TRAIN)")
print(spo_summary)

```

B4: Nhằm thể hiện kết quả so sánh dễ hơn ta sử dụng thư viện **matplotlib** để trực quan hóa việc đo lường chỉ số SPO Loss trung bình và độ sai số trên từng mô hình. Ta thể hiện trên biểu đồ cột với trục tung là SPO trung bình, trục hoành sẽ là các mô hình được

đánh giá, độ sai số SPO Loss sẽ thể hiện dưới dạng thanh. Đầu vào ta sẽ lấy từ kết quả của việc tính toán SPO Loss ở trên.

```
#Thư viện matplotlib

import matplotlib

matplotlib.use("Agg")

import matplotlib.pyplot as plt

# Trực quan hóa SPO Loss

bars = plt.bar(spo_summary["Model"],
spo_summary["Mean_SPO_Loss"],
               yerr=spo_summary["Std_SPO_Loss"], capsize=6,
               color=["#4CAF50", "#2196F3", "#FFC107", "#E91E63"])

plt.title("So sánh chỉ số SPO Loss trung bình theo mô hình",
          fontsize=14)

plt.xlabel("Mô hình", fontsize=12)

plt.ylabel("SPO Loss (Trung bình)", fontsize=12)

plt.grid(axis="y", linestyle="--", alpha=0.6)

plt.tight_layout()

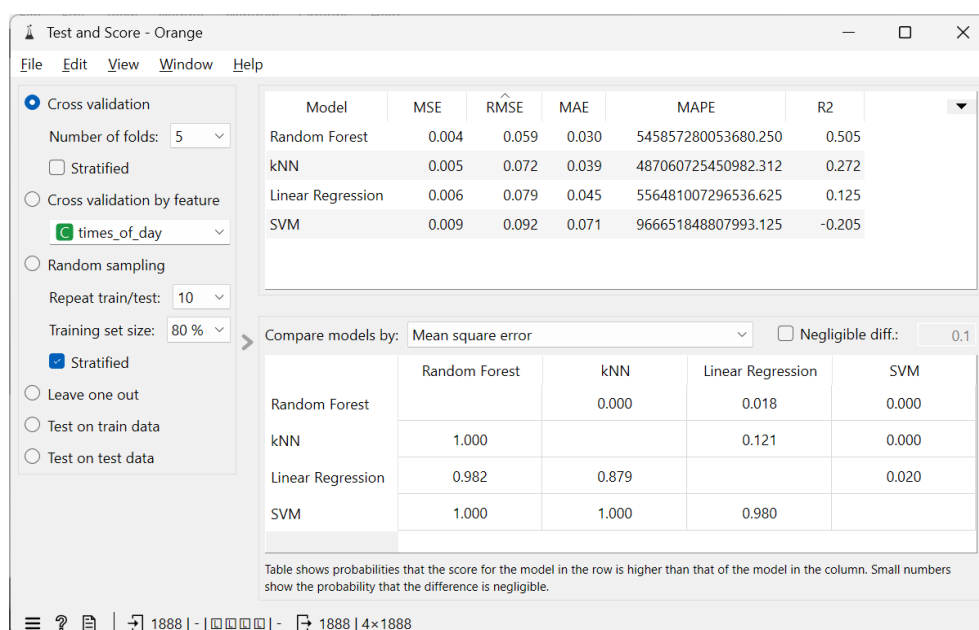
plt.savefig(root2/"SPO_Loss_Chart", dpi=200,
bbox_inches="tight")
```

CHƯƠNG 4: KẾT QUẢ THỰC NGHIỆM

4.1. So sánh hiệu quả của các mô hình học máy và thực hiện ước lượng/ dự báo

4.1.1. Ước lượng/ dự báo với công cụ Orange

Khi thực hiện cross-validation với 5 phần bằng nhau trên train set và so sánh các mô hình huấn luyện bằng công cụ Orange, ta được bảng đánh giá các chỉ số MSE, RMSE, MAE, R2.



Hình 4.1. Bảng kết quả đánh giá mô hình học máy với cross-validation k-fold = 5

Nhận xét: Chỉ số RMSE với các mô hình Random Forest, k-NN, Linear Regression, SVM lần lượt là 0.059, 0.072, 0.079, 0.092; Với MSE lần lượt là 0.004, 0.005, 0.006 và 0.009. Điểm đáng lưu ý là chỉ số R2 của mô hình SVM âm (-0.205), tức là mô hình không có khả năng khái quát được dữ liệu, nếu dự báo sẽ gây ra sai số rất nhiều. Chỉ số càng nhỏ thì mô hình dự báo tốt. Vì thế Random Forest là mô hình tốt nhất cho việc dự báo.

Sau khi đánh giá độ hiệu quả, dự báo click_rate trên tập train set với nhiều mô hình đã được huấn luyện khác nhau.

The screenshot shows the 'Predictions (1) - Orange' window. It displays a table with columns: click_rate, times_of_day, Linear Regression, Random Forest, SVM, kNN, Fold, campaign_id, subject_len, body_len, ean_paragraph_le, and day_of_week. The table contains 20 rows of data. The bottom status bar shows 'Show performance scores' and '1888 | - 1888 | 1888 | -'.

click_rate	times_of_day	Linear Regression	Random Forest	SVM	kNN	Fold	campaign_id	subject_len	body_len	ean_paragraph_le	day_of_week
0.0169016	Noon	0.0795157	0.109085	0.154283	0.0223372	1	24	69	9455	27	5
0.00615558	Evening	0.0420425	0.0247127	0.0465328	0.00594311	1	30	80	11655	22	0
0.005769	Evening	0.0104115	0.00522463	0.0552348	0.00604706	1	31	104	9882	48	0
0.00675312	Evening	-0.00321654	0.030949	0.0323026	0.00566992	1	33	124	11455	27	4
0.00636544	Evening	0.00518168	0.00785449	0.0111038	0.0115763	1	45	87	14431	35	3
0.00329714	Noon	0.049395	0.0103925	0.0228396	0.00513709	1	46	73	11486	42	2
0.00337203	Evening	0.00928356	0.0171034	0.0637615	0.0150348	1	50	85	9717	30	4
0.00853431	Evening	0.00383536	0.00729951	0.0289788	0.165325	1	52	97	10492	39	2
0.00134548	Evening	0.0325575	0.0155821	0.054028	0.144952	1	60	82	5690	86	4
0.0044865	Evening	0.0523673	0.0157899	0.0490699	0.144952	1	64	67	5700	81	4
0.00883873	Evening	0.0330568	0.00757716	0.0459914	0.00704221	1	66	92	16399	23	2
0.00265066	Evening	-0.0283096	0.00545339	-0.00282801	0.0058399	1	68	87	22813	18	2
0.0671835	Evening	0.124928	0.38756	0.352515	0.0924481	1	70	25	1951	160	1
0.035248	Evening	0.0544759	0.0517335	0.124188	0.0489494	1	71	74	9602	23	1
0.0112284	Evening	0.0488171	0.0794598	0.0819886	0.0122782	1	72	100	10176	32	0
0.0034188	Noon	0.0464865	0.00779505	-0.045248	0.0131822	1	74	90	20720	18	5
0.00140631	Evening	-0.0050893	0.00704703	0.00235281	0.00300345	1	77	92	19214	21	3
0.00623635	Evening	-0.000107142	0.00767342	-0.0119722	0.0212435	1	79	94	13412	38	2
0.00203901	Evening	0.0327477	0.00417046	0.0537288	0.00343846	1	100	86	19046	16	1
0.0045904	Noon	0.033882	0.0288373	0.0337784	0.0230517	1	110	74	17353	15	1
0.00489972	Evening	0.028107	0.0106942	0.0443419	0.0230517	1	112	80	17347	15	0

Hình 4.2. Bảng ước lượng CTR với nhiều mô hình khác nhau ở train set với $k\text{-fold} = 5$

Nhận xét: Khi sử dụng cross-validation 5 folds, ta có thể quan sát các dòng đầu:

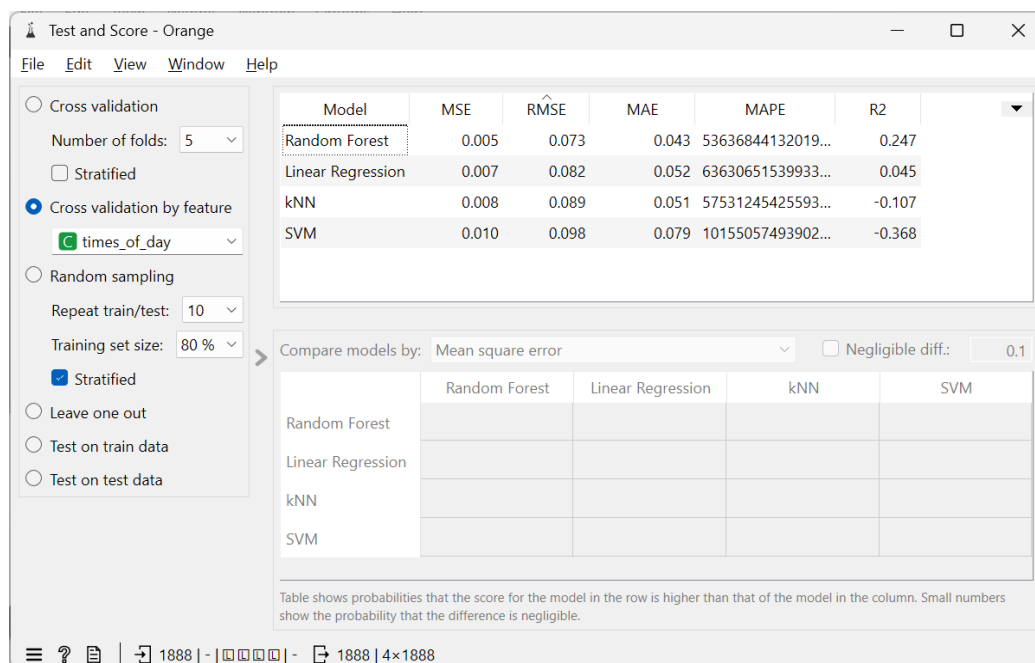
Ở campaign_id 24, CTR thật là 0.0169016, CTR được dự báo ở các mô hình Linear Regression, Random Forest, SVM, k-NN lần lượt là 0.0795757 (dự báo lệch nhiều), 0.0109085 (dự báo gần với giá trị thật), 0.0154283 (gần sát nhất) và 0.0223372 (hơi lệch so với 2 mô hình SVM và k-NN).

Ở campaign_id 30, CTR thật là 0.00615588, CTR được dự báo ở các mô hình Linear Regression, Random Forest, SVM, k-NN lần lượt là 0.040425 (lệch nhiều), 0.0247127 (sát với giá trị thật), 0.0465328 (lệch nhiều) và 0.00594311 (gần sát giá trị thật nhất).

Ta có thể đánh giá tổng quan các mô hình như sau:

- Linear Regression dự báo gần sát giá trị thật với những CTR nhỏ.
- Random Forest giữ độ ổn định khi dự báo gần với giá trị thật như đã thể hiện ở phần đánh giá mô hình.
- SVM dự báo rất xa so với giá trị thật.
- k-NN dự báo cho kết quả ổn định như phần đánh giá mô hình.

Tiếp theo, ta thực hiện đánh giá các mô hình ML bằng cross-validation theo thuộc tính `times_of_day`.



Hình 4.3. Bảng đánh giá các mô hình học máy với cross-validation theo feature `times_of_day`

Nhận xét: Với trường hợp cross-validation theo feature `times_of_day`, ta thấy không có sự khác biệt rõ rệt so với bảng đánh giá các mô hình với cross_validation 5 folds với mô hình tốt nhất là Random Forest (RMSE 0.005). Tuy thế, 2 mô hình k-NN và SVM có chỉ số R2 âm, cho thấy 2 mô hình này dự báo rất lệch so với thực tế.

Một khi đã đánh giá các mô hình ML với cross-validation theo thuộc tính, ta tiến hành ước lượng CTR trên tập train set, và kết quả cho ra ở dưới đây.

The screenshot shows the 'Predictions (1) - Orange' window. It displays a table with columns for 'click_rate', 'times_of_day', and predictions from 'Linear Regression', 'Random Forest', 'SVM', 'kNN', and 'Fold'. The table also includes feature columns: 'campaign_id', 'subject_len', 'body_len', 'ean_paragraph_len', and 'day_of_week'. The data is sorted by 'click_rate' in descending order. The first two rows correspond to the campaigns mentioned in the text.

click_rate	times_of_day	Linear Regression	Random Forest	SVM	kNN	Fold	campaign_id	subject_len	body_len	ean_paragraph_len	day_of_week
0.0108678	Evening	0.0018444	0.0765617	0.0899015	0.0260438	1	4	74	11037	30	4
0.5	Evening	0.12458	0.554143	0.223614	0.320997	1	6	54	2569	256	4

Hình 4.4. Bảng ước lượng CTR với các mô hình học máy khác nhau bằng cross-validation theo feature times_of_day ở train set

Nhận xét: Ta xét 2 campaign đầu bảng ở khung thời gian buổi tối (times_of_day = Evening):

Với campaign_id 4, CTR thật là 0.0108678 cùng với CTR được dự báo của các mô hình là 0.0018444 (Linear Regression), 0.0765617 (Random Forest), 0.0899015 (SVM) và 0.0260438 (k-NN).

Với campaign_id 6, CTR thật là 0.5. Trong khi đó CTR được dự báo của các mô hình là 0.12458 (Linear Regression), 0.554143 (Random Forest), 0.223614 (SVM) và 0.320997 (k-NN)

Ta có thể thấy rõ 4 mô hình đã phản ánh đúng như trong phần đánh giá ở trên khi Random Forest mặc dù ở một vài dòng dữ liệu có sai lệch nhưng vẫn dự báo gần đúng với giá trị thực tế. Đối với Linear Regression thì bị underfitting do độ lệch khi ước lượng rất cao. 2 mô hình SVM và k-NN có kết quả đôi lúc không ổn định, dự báo sai lệch ở một số chiến dịch.

Sau đã huấn luyện các mô hình học máy trên train set, ta tiến hành thực hiện việc dự báo CTR với test set. Kết quả dự báo CTR của nhiều mô hình học máy được thể hiện ở hình dưới đây.

	Random Forest	Linear Regression	SVM	kNN	campaign_id	sender	subject_len	body_len	email_paragraph_len	day_of_week	is_weekend	times_of_day	category
1	0.0999409	0.0876944	0.0926046	0.017925	1889	3	61	12871	11	6	1	Noon	6
2	0.288339	0.176899	0.278124	0.246446	1890	3	54	2569	256	5	1	Morning	2
3	0.268797	0.115761	0.215179	0.183824	1891	3	88	1473	78	4	0	Evening	2
4	0.268797	0.11578	0.215136	0.183824	1892	3	88	1473	78	3	0	Evening	2
5	0.125745	0.0874947	0.122947	0.100719	1893	3	78	9020	29	3	0	Noon	6
6	0.0680828	0.0675641	0.082125	0.0570244	1894	3	99	14983	15	3	0	Noon	6
7	0.0272596	0.0433678	0.120089	0.0591392	1895	3	104	9183	32	1	0	Evening	15
8	0.0778184	0.0660409	0.0817384	0.0397959	1896	3	86	14787	19	0	0	Evening	2
9	0.0887231	0.0790627	0.117653	0.0595107	1897	3	64	12801	16	4	0	Evening	2
10	0.0697605	0.0836899	0.0992027	0.0873548	1898	3	63	11588	28	4	0	Noon	2
11	0.0697605	0.0836979	0.0991837	0.0873548	1899	3	63	11594	28	3	0	Evening	2
12	0.0794924	0.0794892	0.0878593	0.0703228	1900	3	81	9840	22	1	0	Evening	6
13	0.0257578	0.0904318	0.0785503	0.0910374	1901	3	76	9214	51	1	0	Evening	2
14	0.0527955	0.089702	0.0876995	0.0873548	1902	3	80	11655	22	5	1	Noon	2
15	0.0351875	0.0396967	0.0808175	0.0836591	1903	3	80	15553	20	5	1	Noon	15
16	0.038003	0.0282237	0.0595782	0.0836591	1904	3	84	15553	20	3	0	Evening	15
17	0.038003	0.0282422	0.0596614	0.0836591	1905	3	84	15553	20	3	0	Evening	15
18	0.031081	0.0865599	0.115013	0.0192093	1906	3	81	10253	36	2	0	Evening	2
19	0.0712919	0.094191	0.0928827	0.0873548	1907	3	65	11613	25	5	1	Noon	2
20	0.0586107	0.0948913	0.136234	0.0313238	1908	3	67	8405	39	4	0	Noon	2
21	0.0725733	0.084053	0.0932078	0.0777373	1909	3	65	11783	29	3	0	Evening	2
22	0.0608376	0.0874051	0.0859198	0.0507972	1910	3	68	11532	44	2	0	Noon	2

Hình 4.5. Bảng dự báo CTR với nhiều mô hình học máy trên test set

Nhận xét: Random Forest có giá trị dự báo ổn định nhất với độ sai số nhỏ với CTR dao động trong khoảng từ 0.02 đến 0.28, cho thấy mô hình đã phản ứng hợp lý với ngữ cảnh và đặc trưng email của các chiến dịch để đưa ra dự báo chính xác nhất. Với Linear Regression, độ phân bố giá trị dự báo với các chiến dịch gần hơn so với Random Forest nhưng dao động nhỏ hơn (trong khoảng 0.02 đến 0.18), cho thấy rõ mô hình dự báo CTR một cách tuyến tính, khó thể hiện được mối quan hệ phi tuyến giữa đặc trưng email và CTR. Trong khi đó SVM và kNN thể hiện các giá trị dự báo nhỏ hơn so với 2 mô hình còn lại với biên độ dự báo trong khoảng 0.05 đến 0.27, điều này cho thấy 2 mô hình này chưa bắt nhịp tốt với quy luật phi tuyến của mối quan hệ giữa đặc trưng email và CTR.

4.1.2. Dự báo bằng cách sử dụng thư viện Python Scikit-Learn

Bảng kết quả về các chỉ số đo lường hiệu quả của các mô hình học máy (sau khi tiền xử lý dữ liệu):

CV metrics (OOF on TRAIN)					
	Model	RMSE(OOF)	R2(OOF)	MAPE(OOF)	MAE(OOF)
1	RandomForest	0.003357	0.526472	5.606545e+12	0.029648
3	k-NN	0.005424	0.234914	4.443687e+12	0.034890
0	LinearRegression	0.006015	0.151601	5.169075e+12	0.043326
2	SVM(SVR)	0.010600	-0.495106	9.446526e+12	0.088207

Hình 4.6. Bảng kết quả đánh giá các mô hình học máy trên Python

Nhận xét: Kết quả khi đánh giá các mô hình học máy khi sử dụng thư viện của Python (đã qua tiền xử lý dữ liệu) cũng cho ra kết quả gần giống với các bảng kết quả trước đó khi đánh giá trên Orange với chỉ số RMSE của 4 mô hình nằm trong khoảng từ 0.003 cho đến 0.011 và chỉ số R2 ở mô hình SVM cũng ở mức âm.

Sau khi đã đánh giá với các chỉ số kể trên, ta thực hiện việc dự báo CTR cùng với vector chi phí sẽ được sử dụng cho bài toán tối ưu ở trên test set với 4 mô hình:

Mô hình k-NN:

	campaign_id,times_of_day,category,product,cost,value_per_click,pred_ctr,c_pred
1	1890,Morning,2,11,235.45000000000002,1000.0,0.2052629611062967,-205.2629611062967
2	1891,Evening,2,11,197.65,1000.0,0.2660509717316528,-266.0509717316528
3	1892,Evening,2,11,197.65,1000.0,0.26605097173165276,-266.05097173165274
4	1893, Noon,6,26,570.0,1000.0,0.12169284300134238,-121.69284300134238
5	1894, Noon,6,26,878.6500000000001,1000.0,0.09723178102582775,-97.23178102582774
6	1897,Evening,2,11,752.0500000000001,1000.0,0.039948830968615276,-39.948830968615276
7	1898, Noon,2,11,690.9,1000.0,0.08407571070935348,-84.07571070935347
8	1899,Evening,2,11,691.2,1000.0,0.06865128391222414,-68.65128391222413
9	1901,Evening,2,11,578.7,1000.0,0.13509826308209671,-135.0982630820967
10	1902, Noon,2,11,702.75,1000.0,0.07523124144964022,-75.23124144964022
11	1906,Evening,2,11,633.15,1000.0,0.0714425480113348,-71.44254801133481
12	1907, Noon,2,11,693.15,1000.0,0.2591014334882308,-259.1014334882308
13	1908, Noon,2,11,533.75,1000.0,0.14894343590880704,-148.94343590880703
14	1910, Noon,2,11,690.6,1000.0,0.0408191106585473,-40.8191106585473
15	1912,Evening,15,9,548.1,1000.0,0.035168592056391695,-35.168592056391695
16	1919, Noon,1,5,876.4000000000001,1000.0,0.051028829898701514,-51.02882989870152
17	1920,Evening,1,5,447.40000000000003,1000.0,0.10074778720476871,-100.74778720476871
18	1923, Noon,1,5,654.45,1000.0,0.07522112600540878,-75.22112600540878

Hình 4.7. Bảng dự báo CTR và vector chi phí với mô hình k-NN

Mô hình Linear Regression:

	ad_vectorcost_SVM_SVR.csv	test_selected_vectorcost_NeuralNet_MLP.csv	test_selected_vectorcost_LinearRegression.csv
1	campaign_id,times_of_day,category,product,cost,value_per_click,pred_ctr,c_pred		
2	1889, Noon, 6, 26, 754.0500000000001, 1000.0, 0.09701620564014625, -97.01620564014625		
3	1890, Morning, 2, 11, 235.45000000000002, 1000.0, 0.2018070776972739, -201.8070776972739		
4	1891, Evening, 2, 11, 197.65, 1000.0, 0.1066789820194397, -106.6789820194397		
5	1892, Evening, 2, 11, 197.65, 1000.0, 0.10777631665599284, -107.77631665599284		
6	1893, Noon, 6, 26, 570.0, 1000.0, 0.11039439114097846, -110.39439114097846		
7	1894, Noon, 6, 26, 878.6500000000001, 1000.0, 0.0799880214858187, -79.98802148581869		
8	1895, Evening, 15, 9, 591.1500000000001, 1000.0, 0.05976708533587766, -59.76708533587766		
9	1897, Evening, 2, 11, 752.0500000000001, 1000.0, 0.07724571760759556, -77.24571760759557		
10	1898, Noon, 2, 11, 690.9, 1000.0, 0.07912867416077615, -79.12867416077616		
11	1899, Evening, 2, 11, 691.2, 1000.0, 0.06326867575312878, -63.268675753128775		
12	1900, Evening, 6, 26, 612.5, 1000.0, 0.07330395999388817, -73.30395999388817		
13	1901, Evening, 2, 11, 578.7, 1000.0, 0.07751111026648659, -77.5111102664866		
14	1902, Noon, 2, 11, 702.75, 1000.0, 0.10788724452121765, -107.88724452121764		
15	1906, Evening, 2, 11, 633.15, 1000.0, 0.08484452488998076, -84.84452488998076		
16	1907, Noon, 2, 11, 693.15, 1000.0, 0.10135227207662634, -101.35227207662633		
17	1908, Noon, 2, 11, 533.75, 1000.0, 0.1029541053394809, -102.9541053394809		
18	1909, Evening, 2, 11, 701.65, 1000.0, 0.06795622139225249, -67.95622139225249		
19	1910, Noon, 2, 11, 690.6, 1000.0, 0.08363645582963224, -83.63645582963224		

Hình 4.8. Bảng dự báo CTR và vector chi phí với mô hình Linear Regression

Mô hình Random Forest:

	test_selected_vectorcost_RandomForest.csv	test_selected_vectorcost_SVM_SVR.csv	test_selected_vectorcost_LinearRegr
1	campaign_id,times_of_day,category,product,cost,value_per_click,pred_ctr,c_pred		
2	1889, Noon, 6, 26, 754.0500000000001, 1000.0, 0.09816151807846175, -98.16151807846175		
3	1890, Morning, 2, 11, 235.45000000000002, 1000.0, 0.3187204875574241, -318.7204875574241		
4	1891, Evening, 2, 11, 197.65, 1000.0, 0.2547765471630697, -254.7765471630697		
5	1892, Evening, 2, 11, 197.65, 1000.0, 0.2631116322763667, -263.1116322763666		
6	1893, Noon, 6, 26, 570.0, 1000.0, 0.1322114289455957, -132.2114289455957		
7	1896, Evening, 2, 11, 862.35, 1000.0, 0.08366783140849535, -83.66783140849535		
8	1897, Evening, 2, 11, 752.0500000000001, 1000.0, 0.08744442140590448, -87.44442140590448		
9	1898, Noon, 2, 11, 690.9, 1000.0, 0.07784190405051705, -77.84190405051706		
10	1899, Evening, 2, 11, 691.2, 1000.0, 0.07280290536930044, -72.80290536930043		
11	1900, Evening, 6, 26, 612.5, 1000.0, 0.07636058481837776, -76.36058481837776		
12	1902, Noon, 2, 11, 702.75, 1000.0, 0.0717969653556901, -71.7969653556901		
13	1907, Noon, 2, 11, 693.15, 1000.0, 0.07676820140926208, -76.76820140926208		
14	1908, Noon, 2, 11, 533.75, 1000.0, 0.06098180755132937, -60.98180755132937		
15	1909, Evening, 2, 11, 701.65, 1000.0, 0.0706766504737409, -70.6766504737409		
16	1910, Noon, 2, 11, 690.6, 1000.0, 0.06963176758236511, -69.63176758236511		
17	1913, Evening, 1, 5, 638.8, 1000.0, 0.06731275313606017, -67.31275313606017		
18	1914, Evening, 1, 5, 638.8, 1000.0, 0.06780214288478206, -67.80214288478206		
19	1915, Noon, 1, 5, 587.1, 1000.0, 0.08988285626696188, -89.88285626696188		

Hình 4.9. Bảng dự báo CTR và vector chi phí với mô hình Random Forest

Mô hình SVM:

	campaign_id	times_of_day	category	product	cost	value_per_click	pred_ctr	c_pred
1	1889	Noon	6,26	754.0500000000001	1000.0	0.10460178544637089	-104.60178544637088	
2	1890	Morning	2,11	235.45000000000002	1000.0	0.10805758841284707	-108.05758841284707	
3	1891	Evening	2,11	197.65	1000.0	0.10499856836402721	-104.99856836402721	
4	1892	Evening	2,11	197.65	1000.0	0.10492511106256608	-104.92511106256607	
5	1893	Noon	6,26	570.0	1000.0	0.10460740790650036	-104.60740790650036	
6	1895	Evening	15,9	591.1500000000001	1000.0	0.10248596827937739	-102.48596827937739	
7	1897	Evening	2,11	752.0500000000001	1000.0	0.10354506504076355	-103.54506504076355	
8	1898	Noon	2,11	690.9	1000.0	0.10344608727839877	-103.44608727839876	
9	1899	Evening	2,11	691.2	1000.0	0.10291266634616136	-102.91266634616136	
10	1900	Evening	6,26	612.5	1000.0	0.10270067802481109	-102.70067802481108	
11	1901	Evening	2,11	578.7	1000.0	0.10359918849096694	-103.59918849096694	
12	1902	Noon	2,11	702.75	1000.0	0.1051338898457913	-105.1338898457913	
13	1906	Evening	2,11	633.15	1000.0	0.10346925678849062	-103.46925678849061	
14	1907	Noon	2,11	693.15	1000.0	0.10519822892291508	-105.19822892291508	
15	1908	Noon	2,11	533.75	1000.0	0.10478634987021807	-104.78634987021806	
16	1909	Evening	2,11	701.65	1000.0	0.10350843412548472	-103.50843412548471	
17	1910	Noon	2,11	690.6	1000.0	0.10336826225381401	-103.36826225381401	
18	1911	Evening	15,9	591.1500000000001	1000.0	0.10189104452483122	-101.89104452483122	

Hình 4.10. Bảng dữ liệu báo CTR và vector chi phí với mô hình SVM

4.2. Kết quả lựa chọn chiến dịch sau tối ưu hóa từ dự báo/ ước lượng

Với việc chọn các chiến dịch có khả năng mang lại giá trị kinh tế cao, sau khi đã dự đoán tập test với từng mô hình học máy thì sau khi thực hiện MILP đã chọn ra tập các chiến dịch tối ưu nhất định từ tập test có 762 campaign từ các ràng buộc cài đặt mặc định qua user input (Giả định giá trị kinh tế mỗi chiến dịch $V = 1000$, ngân sách chi không quá 35%, không có ràng buộc về hạn mức theo buổi).

```

=== Thiết lập tham số kinh tế & quota ===
Giá trị mỗi click (V_CONST), mặc định là [1000.0]:
Nếu thiếu cột chi phí thì tạo proxy từ body_len? (yes/no), mặc định [yes]:
Ngân sách nhập kiểu (ratio / absolute / none), mặc định là [ratio]:
Nhập tỷ lệ ngân sách (mặc định 0.35 = 35%) [0.35]:
Quota tối đa cho slot morning (trống = bỏ qua) []:
Quota tối đa cho slot noon (trống = bỏ qua) []:
Quota tối đa cho slot evening (trống = bỏ qua) []:

```

Hình 4.11. Cài đặt các ràng buộc bằng cách cho người dùng nhập input

Sau khi nhập ràng buộc xong thì bảng kết quả tổng quan từ tối ưu tập chiến dịch sẽ hiện ra, ta thấy được rằng ước lượng từ mô hình SVM không thể dùng được cho bài toán tối ưu MILP cho nên ta thay thế bằng Knapsack để thực hiện tối ưu.

Optimization (vector cost) on TEST					
	Model	Used_MILP	...	Chosen	ObjectiveValue(c_pred^T x_pred)
0	LinearRegression	True	...	369	-34266.860056
1	RandomForest	True	...	344	-37709.885469
2	SVM(SVR)	False	...	385	-39812.154739
3	k-NN	True	...	337	-30364.478116

Hình 4.12. Kết quả tổng quan từ tối ưu tập chiến dịch

Với **Linear Regression**, sau tối ưu có 369 campaign được chọn, tỷ lệ tập chiến dịch được chọn trên test set là 48.43%.

Selected items (TEST) - LinearRegression							
	campaign_id	times_of_day	category	...	value_per_click	pred_ctr	c_pred
0	1889	Noon	6	...	1000.0	0.097016	-97.016206
1	1890	Morning	2	...	1000.0	0.201807	-201.807078
2	1891	Evening	2	...	1000.0	0.106679	-106.678982
3	1892	Evening	2	...	1000.0	0.107776	-107.776317
4	1893	Noon	6	...	1000.0	0.110394	-110.394391
..	...	...	...	...	...	...	...
364	2646	Evening	15	...	1000.0	0.070237	-70.236977
365	2647	Evening	2	...	1000.0	0.107941	-107.940702
366	2648	Evening	15	...	1000.0	0.069786	-69.786397
367	2649	Noon	2	...	1000.0	0.142833	-142.832525
368	2650	Evening	2	...	1000.0	0.093838	-93.838117

[369 rows x 8 columns]

Hình 4.13. Bảng thể hiện tập các chiến dịch tối ưu được chọn sau khi dự báo bằng mô hình Linear Regression

Random Forest: sau tối ưu có 344 campaign được chọn, tỷ lệ tập chiến dịch được chọn trên test set là 45.14%.

Selected items (TEST) - RandomForest							
	campaign_id	times_of_day	category	...	value_per_click	pred_ctr	c_pred
0	1889	Noon	6	...	1000.0	0.098162	-98.161518
1	1890	Morning	2	...	1000.0	0.318720	-318.720488
2	1891	Evening	2	...	1000.0	0.254777	-254.776547
3	1892	Evening	2	...	1000.0	0.263112	-263.111632
4	1893	Noon	6	...	1000.0	0.132211	-132.211429
..	...	...	...	...	...	...	...
339	2639	Evening	10	...	1000.0	0.098023	-98.022913
340	2641	Morning	2	...	1000.0	0.306964	-306.964068
341	2643	Evening	2	...	1000.0	0.244714	-244.714236
342	2645	Evening	2	...	1000.0	0.270457	-270.457105
343	2649	Noon	2	...	1000.0	0.284353	-284.353021
[344 rows x 8 columns]							

Hình 4.14. Bảng thể hiện tập các chiến dịch tối ưu được chọn sau khi dự báo bằng mô hình Random Forest

SVM: sau tối ưu có 385 campaign được chọn, tỷ lệ tập chiến dịch được chọn trên test set là 50.52%.

Selected items (TEST) - SVM(SVR)							
	campaign_id	times_of_day	category	...	value_per_click	pred_ctr	c_pred
0	1889	Noon	6	...	1000.0	0.104602	-104.601785
1	1890	Morning	2	...	1000.0	0.108058	-108.057588
2	1891	Evening	2	...	1000.0	0.104999	-104.998568
3	1892	Evening	2	...	1000.0	0.104925	-104.925111
4	1893	Noon	6	...	1000.0	0.104607	-104.607408
..	...	...	...	...	...	...	...
380	2646	Evening	15	...	1000.0	0.102497	-102.496907
381	2647	Evening	2	...	1000.0	0.104344	-104.343779
382	2648	Evening	15	...	1000.0	0.102760	-102.760481
383	2649	Noon	2	...	1000.0	0.105622	-105.621933
384	2650	Evening	2	...	1000.0	0.103331	-103.330693
[385 rows x 8 columns]							

Hình 4.15. Bảng thể hiện tập các chiến dịch tối ưu được chọn sau khi dự báo bằng mô hình SVM

k-NN: có 337 campaign sau tối ưu được chọn, tỷ lệ tập chiến dịch được chọn trên test set là 44.23%.

.

Selected items (TEST) - k-NN							
	campaign_id	times_of_day	category	...	value_per_click	pred_ctr	c_pred
0	1890	Morning	2	...	1000.0	0.205263	-205.262961
1	1891	Evening	2	...	1000.0	0.266051	-266.050972
2	1892	Evening	2	...	1000.0	0.266051	-266.050972
3	1893	Noon	6	...	1000.0	0.121693	-121.692843
4	1894	Noon	6	...	1000.0	0.097232	-97.231781
..	...	...	...	...	...	...	...
332	2643	Evening	2	...	1000.0	0.240960	-240.960063
333	2644	Evening	2	...	1000.0	0.085103	-85.102975
334	2645	Evening	2	...	1000.0	0.170997	-170.996598
335	2647	Evening	2	...	1000.0	0.082792	-82.791953
336	2649	Noon	2	...	1000.0	0.170997	-170.996598

[337 rows x 8 columns]

Hình 4.16. Bảng thể hiện tập các chiến dịch tối ưu sau khi dự báo bằng mô hình k-NN

4.3. Phân tích hiệu quả quyết định bằng hàm mất mát SPO Loss

Kết quả đo lường các chỉ số SPO Loss bao gồm trung bình, trung vị, và giá trị chuẩn theo từng fold và theo các mô hình với toàn cục.

4.3.1. Nhận xét trên từng fold

```
Model,Fold,UsedMILP(True),UsedMILP(Pred),True_Objective,Objective_under_true_of_x_pred,SPO_Loss
LinearRegression,1,True,True,-12302.454322045827,-10723.155374042377,1579.2989480034503
LinearRegression,2,True,True,-14473.062746905425,-12703.529296316396,1769.5334505890296
LinearRegression,3,True,True,-13731.021069025644,-11805.686727172715,1925.3343418529294
LinearRegression,4,True,True,-14225.757570385938,-12470.635752374988,1755.12181801095
LinearRegression,5,True,True,-16961.53547027637,-15346.990071180902,1614.5453990954666
RandomForest,1,True,True,-12302.454322045827,-11538.350121905156,764.1042001406713
RandomForest,2,True,True,-14473.062746905425,-13201.531785443596,1271.530961461829
RandomForest,3,True,True,-13731.021069025644,-12778.556493254147,952.464575771497
RandomForest,4,True,True,-14225.757570385938,-12574.252246451142,1651.5053239347963
RandomForest,5,True,True,-16961.53547027637,-15925.980065852855,1035.5554044235141
SVM(SVR),1,True,False,-12302.454322045827,-10554.33417457834,1748.1201474674872
SVM(SVR),2,True,True,-14473.062746905425,-12335.653463296927,2137.4092836084983
SVM(SVR),3,True,True,-13731.021069025644,-11306.545007201617,2424.476061824027
SVM(SVR),4,True,True,-14225.757570385938,-10991.957974420939,3233.7995959649998
SVM(SVR),5,True,True,-16961.53547027637,-13627.772519095135,3333.762951181234
k-NN,1,True,True,-12302.454322045827,-10948.565047988242,1353.8892740575848
k-NN,2,True,True,-14473.062746905425,-12796.673291683124,1676.389455222301
k-NN,3,True,True,-13731.021069025644,-12059.307877111041,1671.7131919146032
k-NN,4,True,True,-14225.757570385938,-12860.158164476623,1365.5994059093155
k-NN,5,True,True,-16961.53547027637,-15469.542872401413,1491.9925978749561
```

Hình 4.17. Bảng đo lường SPO Loss với từng mô hình trên từng fold

Mô hình Linear Regression có SPO Loss nằm trong miền giá trị (1519, 1925), điều đó chứng tỏ mỗi quyết định tối ưu từ dự báo của mô hình này sẽ gây ra giá trị tổn thất đáng kể.

Mô hình Random Forest có SPO Loss nằm trong miền giá trị (764, 1651) thể hiện mỗi quyết định tối ưu gần sát với quyết định oracle nhất, dẫn đến thiệt hại về kinh tế thấp.

SPO Loss của SVM nằm trong khoảng từ 1748 đến 3333 nghĩa là việc tối ưu hóa từ dự báo SVM bị sai nhiều dẫn đến tổn thất rất lớn.

Chỉ số SPO Loss từ quyết định của k-NN nằm trong mức từ 1353 cho đến 1676, tổn thất thấp hơn Linear Regression và SVM, nhưng cao hơn Random Forest.

4.3.2. Nhận xét trên toàn cục

```
Model,Mean_SPO_Loss,Median_SPO_Loss,Std_SPO_Loss,Folds_Done
RandomForest,1135.0320931464616,1035.5554044235141,341.39074056781755,5
k-NN,1511.916784995752,1491.9925978749561,157.610749779391,5
LinearRegression,1728.7667915103652,1755.12181801095,138.18689562487913,5
SVM(SVR),2575.5136080092493,2424.476061824027,690.581310622288,5
```

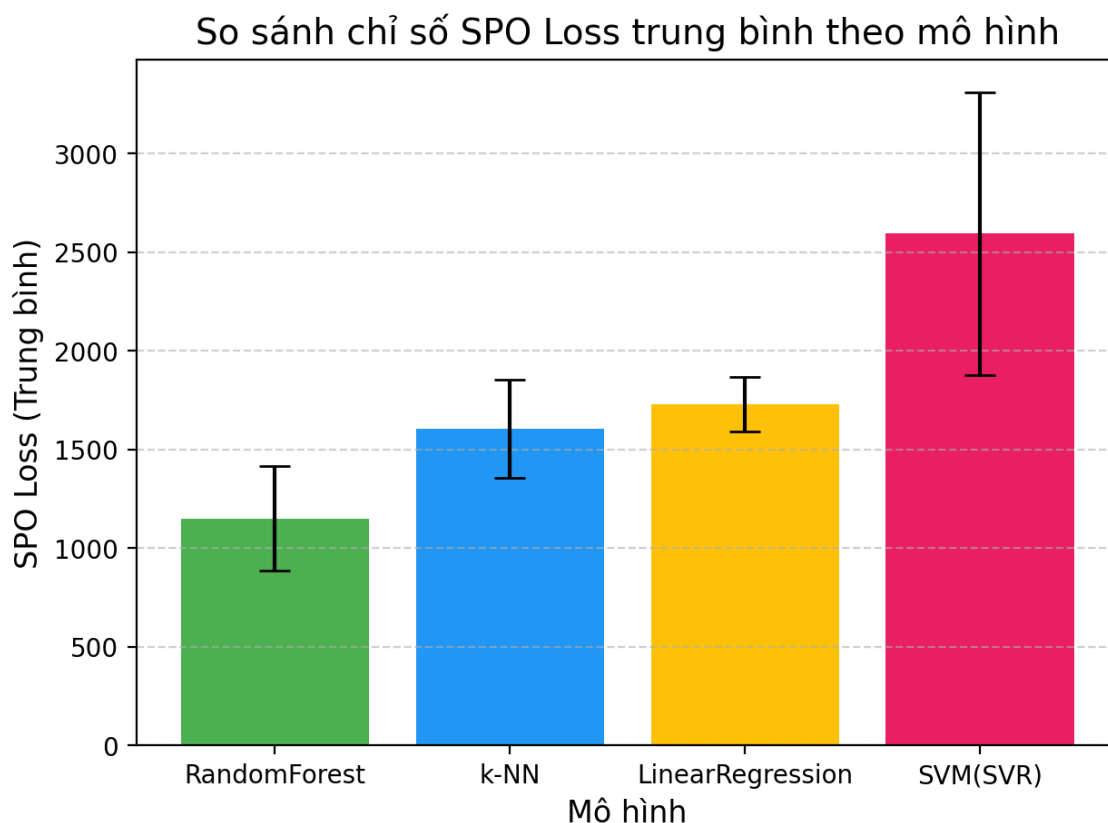
Hình 4.18. Bảng đo lường SPO Loss với từng mô hình trên toàn cục

Trên toàn cục, mô hình Random Forest có tổn thất trung bình trên mỗi campaign thấp nhất (~1135) và thấp hơn so với các mô hình k-NN (~1511), Linear Regression (~1728), SVM (~2575). Đối với tổn thất trung vị cũng thể hiện Random Forest cũng thấp nhất (~1035), so với các mô hình khác thì SVR vẫn có tổn thất trung vị cao nhất (~2424). Độ lệch chuẩn của SPO Loss của 4 mô hình ở trong mức khoảng lần lượt là 341 (Random Forest), 157 (k-NN), 138 (Linear Regression), 690 (SVM).

Như vậy Random Forest có hiệu quả nhất trong việc đưa ra quyết định tối ưu gần với quyết định oracle. Mô hình k-NN có SPO Loss cao hơn nhưng vẫn ổn định với độ lệch chuẩn thấp, vẫn đáng tin cậy khi đưa ra quyết định tối ưu. Linear Regression có tổn thất cao hơn k-NN nhưng vẫn có tính ổn định với độ lệch chuẩn thấp. SVM là mô hình có độ lệch chuẩn lớn nhất và tổn thất SPO Loss cao nhất, gây nhiều rủi ro cho việc đưa ra các quyết định tối ưu từ dự báo.

4.3.3. Kết quả trực quan hóa so sánh SPO Loss trên từng mô hình

Biểu đồ cột thể hiện chỉ số SPO với trục X (trục hoành) là 4 mô hình được đánh giá gồm Random Forest, k-NN, Linear Regression và SVM (SVR), trục tung là SPO Loss trung bình của từng mô hình, giá trị càng thấp thì mô hình gây ra giá trị tổn thất ít hơn, chứng tỏ mô hình tốt.



Hình 4.19: Biểu đồ cột thể hiện so sánh chỉ số SPO Loss trung bình theo mô hình học máy (error bar = Std SPO Loss)

Mô hình Random Forest là mô hình có tính ổn định cao, cho ra quyết định tối ưu dự báo gần với quyết định tối ưu thật với chỉ số SPO Loss trung bình ở mức thấp nhất so với 3 mô hình còn lại, độ sai số SPO Loss của mô hình này ở trong mức chấp nhận được. Mô hình k-NN có SPO Loss trung bình cao hơn Random Forest, nhưng độ sai số tổn thất kinh tế tương đương. Mô hình Linear Regression có SPO Loss trung bình cao nhưng có độ sai số ít nhất trong các mô hình. SPO Loss của SVM (SVR) gây ra tổn thất kinh tế cao nhất với SPO trung bình và độ sai số lớn.

Qua đó, Random Forest là mô hình dự báo phù hợp nhất để giảm thiểu tổn thất kinh tế từ sai lệch so với quyết định thật, đảm bảo ổn định trong việc ra quyết định.

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Đóng góp của đề tài

Qua một quá trình ước lượng/ dự báo sau đó tìm ra tập chiến dịch tối ưu rồi đánh giá chúng với SPO Loss, kết quả cuối cùng cho thấy Random Forest là tốt nhất so với các mô hình còn lại khi thể hiện độ sai số thấp trên các chỉ số truyền thống lẫn giá trị tổn thất tối ưu sau dự báo. Khẳng định này chỉ đúng khi xét tới bộ dữ liệu được cung cấp. Khi sử dụng bộ dữ liệu khác có thay đổi các tính chất liên quan đến ràng buộc, đặc trưng của thuộc tính thì cách tiếp cận SPO vẫn đúng khi thay đổi các tham số phù hợp với bộ dữ liệu. Mặc dù đã có những bài báo đề cập đến việc dự báo CTR bằng các mô hình học máy, đề tài này cung cấp một cái nhìn mới về việc sử dụng dự báo cho công việc tối ưu, từ đó đánh giá quyết định sau tối ưu bằng chỉ số SPO Loss ngoài những chỉ số đánh giá truyền thống. Việc so sánh với nhiều mô hình học máy có tính hồi quy như Linear Regression, Random Forest, SVR, k-NN giúp sàng lọc được mô hình tốt cho việc ra quyết định.

5.2. Hạn chế và hướng phát triển

5.2.1. Hạn chế

Trong quá trình nghiên cứu về khung lý thuyết SPO, bài báo được nêu của (Elmachtoub & Grigas, 2017) mang tính hàn lâm, mức độ hiểu rất cao, quá sức với sinh viên để có thể áp dụng toàn bộ khung. Vì thế SPO Loss được đề cập trong đề tài này chỉ mang tính ứng dụng và không đưa SPO+ vào huấn luyện; SPO Loss chỉ được đo lường trên tập train-CV mà không đo lường trên tập test set.

Ngoài ra bộ dữ liệu chỉ cung cấp các đặc trưng của email để phục vụ cho việc dự báo, đối với bài toán tối ưu công việc tìm ràng buộc cần lấy chi tiết từ các features trong bộ dữ liệu và nghiên cứu mối liên hệ với hàm mục tiêu.

5.2.2. Hướng phát triển

Trong tương lai, các nghiên cứu trong đề án này có thể phát triển theo nhiều hướng khác nhau. Khung lý thuyết SPO chưa phát huy được hết tiềm năng do giới hạn về mặt học thuật do vậy có thể mở rộng khung theo hướng phù hợp hơn cho việc lập trình, đo lường chỉ số hơn. Ngoài ra tập dữ liệu có thể được mở rộng với các thuộc tính khác (tỷ lệ chuyển đổi, các yếu tố tương tác đa kênh, đa phương tiện trong email,...) nhằm giúp cho mô hình có thể nắm bắt dễ hơn tính chất của dữ liệu, phản ánh chính xác về hành vi mở email của khách hàng. Đối với vấn đề ước lượng/ dự báo, ngoài các mô hình học máy quen thuộc, ta có thể sử dụng các mô hình học máy nâng cao và phức tạp hơn (Neural Network, XGBoost, Gradient Boosting) nhằm tăng tính khách quan của việc dự báo. Ở tầng tối ưu hóa, ta chỉ mới xem xét ràng buộc về hạn mức khung giờ gửi, ngân sách. Trong thực tế, doanh nghiệp luôn phải đầu đầu với các ràng buộc khác như chi phí nhân sự, mua cơ sở vật chất phục vụ cho Email Marketing, yêu cầu về mặt pháp lý. Mở rộng MILP đối với những ràng buộc này hay áp dụng những phương pháp tối ưu khác sẽ giúp giải quyết được bài toán tối ưu ở quy mô lớn hơn và phức tạp hơn. Một hướng đi mang tính ứng dụng và xu hướng là áp dụng SPO vào trong các hệ thống CRM hay tự động hóa với AI/ Marketing Automation giúp việc dự báo, tối ưu hóa và đo lường tổn thất diễn ra trong thời gian thực, tạo ra một chu trình học hỏi liên tục.

TÀI LIỆU THAM KHẢO

- Abebe, T. H. (2024). Regression Analysis: a theoretical approach. *Journal of Statistical and Econometric Methods*, 1–40. <https://doi.org/10.47260/jsem/1311>
- Akçay, Y., Li, H., & Xu, S. H. (2006). Greedy algorithm for the general multidimensional knapsack problem. *Annals of Operations Research*, 150(1), 17–29. <https://doi.org/10.1007/s10479-006-0150-4>
- Awad, M., & Khanna, R. (2015). Efficient learning machines. In *Apress eBooks*. <https://doi.org/10.1007/978-1-4302-5990-9>
- Bawm, Z. L., & Nath, R. P. D. (2014). A Conceptual Model for effective email marketing. *IEEE*. <https://doi.org/10.1109/iccitechn.2014.7073103>
- Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32. <https://doi.org/10.1023/a:1010933404324>
- Dantzig, G. B. (1957). Discrete-Variable extremum problems. *Operations Research*, 5(2), 266–288. <https://doi.org/10.1287/opre.5.2.266>
- Demšar, J., Curk, T., Erjavec, A., Gorup, Č., Hočevár, T., Milutinovič, M., Možina, M., Polajnar, M., Toplak, M., Starič, A., Štajdohar, M., Umek, L., Žagar, L., Žbontar, J., Žitnik, M., & Zupan, B. (2013). Orange: data mining toolbox in python. *Journal of Machine Learning Research*, 14(1), 2349–2353. <http://eprints.fri.uni-lj.si/2267/1/2013%2DDemsar%2DOrange%2DJMLR.pdf>

Elmachtoub, A. N., & Grigas, P. (2017). Smart “Predict, then Optimize.” *arXiv (Cornell University)*. <https://doi.org/10.48550/arxiv.1710.08005>

El-Shaarawi, A. H., & Piegorisch, W. W. (2002). *Encyclopedia of Environmetrics*. John Wiley & Sons.

GOBRANDING. (2025, January 3). CTR là gì? Chỉ số CTR bao nhiêu là tốt? Cách tối ưu hiệu quả. Công ty Marketing Online GOBRANDING. <https://gobranding.com.vn/ctr-la-gi-chi-so-ctr-bao-nhieu-la-tot/>

Gurobi Optimization. (2025, September 25). *Mixed-Integer Programming (MIP/MILP) – A primer on the basics - Gurobi Optimization*. <https://www.gurobi.com/resources/mixed-integer-programming-mip-a-primer-on-the-basics/>

Han, J., Kamber, M., & Pei, J. (2012). Data mining: concepts and techniques. *Choice Reviews Online*, 49(06), 49–3305. <https://doi.org/10.5860/choice.49-3305>

Hancock, J. T., & Khoshgoftaar, T. M. (2020). Survey on categorical data for neural networks. *Journal of Big Data*, 7(1). <https://doi.org/10.1186/s40537-020-00305-w>

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95. <https://doi.org/10.1109/mcse.2007.55>

Kohavi, R. (1995). A study of cross-validation and bootstrap for accuracy estimation and model selection. *International Joint Conference on Artificial Intelligence*, 2, 1137–1143. <http://ijcai.org/Proceedings/95-2/Papers/016.pdf>

Mejtoft, T., Hedlund, J., Cripps, H., Söderström, U., & Norberg, O. (2021). Designing Call to Action: Users' Perception of Different Characteristics. *Digital Support From Crisis to Progressive Change*. <https://doi.org/10.18690/978-961-286-485-9.30>

milp — *SciPy* v1.16.2 *Manual*. (n.d.).
<https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.milp.html>

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). *SciKit-Learn: Machine Learning in Python*. <https://jmlr.csail.mit.edu/papers/v12/pedregosa11a.html>

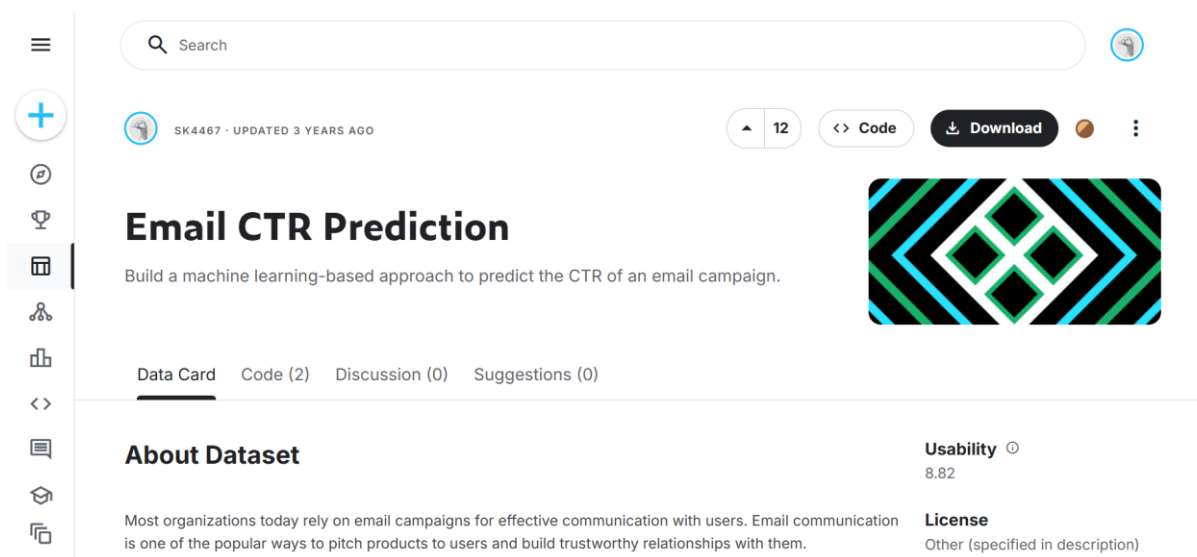
Tang, B., & Khalil, E. B. (2022). PyEPO: a PyTorch-based End-to-End Predict-then-Optimize library for linear and integer programming. *arXiv (Cornell University)*.
<https://doi.org/10.48550/arxiv.2206.14234>

Wu, J., Li, K. J., & Liu, J. S. (2016). Bayesian inference for assessing effects of email marketing campaigns. *Journal of Business and Economic Statistics*, 36(2), 253–266.
<https://doi.org/10.1080/07350015.2016.1141096>

PHỤ LỤC A: HƯỚNG DẪN SỬ DỤNG BỘ DỮ LIỆU KAGGLE

B1: Truy cập vào trang web chứa bộ dữ liệu Kaggle

<https://www.kaggle.com/datasets/sk4467/email-ctr-prediction>



Hình A.1. Giao diện trang web chứa bộ dữ liệu Kaggle

B2: Tải file zip chứa 2 file csv (bao gồm train set và test set)

B3: Giải nén file zip

B4: Chép 2 file đã tải vào thư mục để chứa bộ dữ liệu phục vụ cho đầu vào của source code trong khóa luận.

PHỤ LỤC B: HƯỚNG DẪN TÁI LẬP THỰC NGHIỆM BẰNG PYCHARM

1. Môi trường chạy:

- **Hệ điều hành:** Windows 11
- **Ứng dụng sử dụng:** Pycharm version 2025.1
- **Phiên bản môi trường Python:** 3.9 (cần thiết để chạy được SciKit, SciPy ổn định)
- **Phần cứng:** RAM 12GB, CPU 4 Cores
- **Thư viện Python chính**
 - `numpy==2.0.2`
 - `pandas==2.2.3`
 - `scikit-learn==1.6.1`
 - `scipy==1.13.1` (có `scipy.optimize.milp`)
 - `matplotlib==3.9.4`
 - `joblib==1.5.1`

2. Cấu trúc thư mục dự án:

Đề án khóa luận/

├── email_ctr_prediction/

| ├── test_Data.csv

| └── train_data.csv

├── out/

| └── ctr_predictions.csv

```

|   ├── cv_metrics.csv
|   ├── opt_vector_cost_test.csv
|   ├── spo_loss_train_folds.csv
|   ├── spo_loss_train_summary.csv
|   ├── test_selected_vectorcost_k-NN.csv
|   ├── test_selected_vectorcost_LinearRegression.csv
|   ├── test_selected_vectorcost_RandomForest.csv
|   └── test_selected_vectorcost_SVM_SVR.csv
└── src/
    └── SPO.py

```

3. Thiết lập output dữ liệu

- Chép file train set và test set qua thư mục email_ctr_prediction.
- Mở bộ dữ liệu trong pycharm để xem thông tin cột và dòng.

4. Chạy thực nghiệm

- Mở file SPO.py -> Run SPO.py hoặc nhấn Ctrl + F5
- Thiết lập các tham số để thực hiện các bài toán
 - Giá trị mỗi click: điền một giá trị số
 - Tạo proxy từ body_len với cột chi phí ngân sách cần bỏ ra: nhập yes hoặc no (viết thường)
 - Ngân sách: nhập kiểu ratio/ absolute/ none
 - Nếu nhập ratio: nhập một số thập phân trong khoảng từ 0 đến 1 (VD: 0.35)

- Nếu nhập absolute: nhập một số thực bất kì
- Nếu nhập none: không có ràng buộc ngân sách
 - Quota tối đa cho slot morning, noon và evening: nhập số nguyên dương
- Nếu nhập đúng định dạng và chạy thành công, các file kết quả sẽ được sinh ra/ ghi đè trong đường dẫn Đồ án khóa luận/out bao gồm các file `cv_metrics.csv`, `ctr_predictions.csv`, `opt_vector_cost_test.csv`, `test_selected_vectorcost_*.csv` (mỗi mô hình một file) và `spo_loss_train_folds.csv`, `spo_loss_train_summary.csv`

5. Kiểm tra kết quả

- Mở các file csv trong đường dẫn /out bằng Pycharm để xem nhanh
- Kiểm tra các file `cv_metrics.csv`, `ctr_predictions.csv`, `opt_vector_cost_test.csv`, `test_selected_vectorcost_*.csv`, `spo_loss_train_folds.csv`, `spo_loss_train_summary.csv` đã có đầy đủ số liệu chưa.

6. Lỗi thường gặp và cách xử lý

- **FileNotFoundError**: kiểm tra lại đường dẫn của bộ dữ liệu xem đã tham chiếu tới source code chưa.
- **MAPE cao/ R2 âm**: đã giải thích trong đồ án.
- **MILP không hội tụ**: giảm kích thước test, nới `time_limit`, kiểm tra ràng buộc budget/quota.

7. Link GitHub để tiến hành thực nghiệm:

<https://github.com/HackJacker073/Khoa-luan-tot-nghiep-SmartPredictThenOptimize>